SWE 216

Summary

Done By
Maryam Aladsani
s2022***

Table of Contents

(1) Why Requirements Engineering (RE)? *3*
(2) Introduction to Requirements *6*
(3) How to Write Good Requirements *13*
(4) RE in Software Development Life Cycles *18*
(5) System Vision, Context and RE Framework *24*
(6) Requirements Discovery *30*
(7) Fundamentals of Scenarios *44*
(8) Solution Oriented Requirements *56*
(9) Requirements Prioritization and Agile Estimation *65*
(10) Requirements Conflicts and Negotiation *77*
(11) Requirements Validation *84*
(12) Fundamentals of Requirements Management *92*
(13) Prototyping *101*
(14) Usability Requirements *105*

Note: All content is taken directly from the slides.

(1) Why Requirements Engineering (RE)?

A Requirement is:

- Capturing the purpose of a system.
- An expression of the ideas to be embodied in the system or application under development.
- A statement about the proposed system that all stakeholders agree upon in order to solve the customer's problem.
- A statement which translates or expresses a need and its associated constraints and conditions [IEEE 29148-2011].

Requirement Engineering (RE) is:

- The activity of development, elicitation, specification, analysis, and management of the stakeholder requirements, which are to be met by a new or evolving system.

- **Standard definition:** RE is the interdisciplinary function that mediates between the domains of the acquirer and supplier to establish and maintain the requirements to be met by the system, software, or service of interest. RE is concerned with discovering, eliciting, developing, analyzing, determining verification methods, validating, communicating, documenting, and managing requirements [IEEE 29148].

Six factors must be met for a project to be successful

1. It's delivered on time.
2. Its cost doesn't exceed its budget.
3. It works as designed.
4. People use it.
5. The people who funded the project are happy with it.
6. It meets the goals that drove the project.

SCHAOS Report 2015

- Produced by the Standish Group.
- Investigated software projects success and failure.
    - Successful projects are
        - ✓ Resolved within a reasonable estimated time (OnTime).
        - ✓ Stayed within budget (OnBudget).
        - ✓ Delivered customer and user satisfaction (OnGoal, Value, Satisfaction).
    - Challenged projects are
        - ✓ Late.
        - ✓ Over budget (or both late and over budget).
        - ✓ Less than satisfied customers (did not meet their expectations)
    - Failed projects are either Cancelled before it gets resolved or is resolved but never get used.

- The larger the project, the less valuable the return rate. In many cases larger projects never return value to an organization.

- The faster the projects go into production the quicker the payback starts to accumulate.

- Complex Projects have both the highest challenged and failed rates, Complexity is often caused by size, conflicting goals, and large budgets.
- Restarts is one of the major causes of both cost and time overruns.
    - For every 100 projects that start, there are 94 restarts.
    - Some projects can have several restarts.

- The spectrum of failures ranges from failures with minor consequences to failure with disastrous effects.

(2) Introduction to Requirements

User vs. System Requirements

- ***User* requirements**
  - Statements in natural language plus diagrams of the services the system provides and its operational constraints.
  - Written for clients.

- ***System* requirements**
  - A structured document setting out detailed descriptions of the system's functions, services, and operational constraints.
  - Defines what should be implemented so may be part of a contract between client and contractor.

- User requirement may be expanded into several system requirements.

- **Why do we have different types of requirements?**

  Different levels of requirements are useful because they communicate information about the system to different types of readers.

Domain Requirements

- Derived from the application domain.
- Describe system characteristics and features that reflect the domain.
- May be new functional requirements, constraints on existing requirements, or define specific computations.
- If domain requirements are not satisfied, the system may be unworkable.

Domain requirements problems

- **Understandability:** Requirements are expressed in the language of the application domain; This is often not understood by software engineers developing the system.

- **Implicitness/Tacit knowledge:** Domain specialists understand the area so well that they do not think of making the domain requirements explicit. People are often unaware of the tacit knowledge they possess and therefore cannot express it to others "It's obvious", "Assume".

Functional Requirements (FR)

Define services the system should provide, behavior of the system and in some cases also what the system should not do.

Example: *A functional requirement defined for a security system of a building:*
R1: If a sensor detects that a glass pane is damaged or broken, the system **shall inform** the security company.

Impact of Functional Requirements on Project Success:

- **Scope and Deliverables:** Functional requirements define what the project is supposed to achieve. Failure to define these clearly can lead to scope creep, where the project expands beyond its original purpose, causing delays and budget overruns.
- **User Satisfaction:** Failure to meet these requirements can result in customer dissatisfaction and decreased adoption, which can harm the project's success.
- **Project Timeline:** Delays in meeting functional requirements can directly impact the project timeline. If key features are not delivered on time, it can lead to project delays, missed deadlines, and increased costs.
- **Resource Allocation:** Functional requirements dictate the resources (e.g., people, technology, materials) needed for the project. Insufficient resources allocated to meet these requirements can lead to quality issues and project failure.

Non-Functional Requirements (NFR)

Non-Functional Requirements Classifications:

- **Product requirements** - system must behave in a particular way , e.g., execution speed, memory usage, usability requirements, reliability requirements (set out the acceptable failure rate), etc.
- **Organizational requirements** – derived from organizational policies and procedures, e.g., process standards used, implementation requirements, development environment, etc.
- **External requirements** - factors which are external to the system, e.g., Regulatory requirements, interoperability requirements, legislative requirements, etc.

Non- Functional Requirements refer to **Quality** and **Constraints**.

Quality requirements: Define quality properties of the system, a component, a service or a function (e.g., performance).

Quality Requirements (Important to users):

- Availability: refers to the percentage of time during which the system is actually available for use and fully operational.
- Efficiency: is a measure of how well the system utilizes hardware resources such as processor time, memory, or communication bandwidth.
- Flexibility: indicates how much effort is needed to extend the system with new capabilities.
- Integrity: denotes how well the system is protected against unauthorized access, violations of data privacy, information loss, and infections through maleficent software.
- Interoperability: indicates how easily the system can exchange data or services with other systems.
- Reliability: is the probability of the system executing without failure for a specific period.
- Robustness: is the degree to which a system or component continues to function correctly when confronted with invalid inputs, defects in connected systems or components, or unexpected operating conditions.
- Usability: measures the effort the user requires to prepare input for, operate, and interpret the output of the system.

Quality Requirements (Important to developers):

- Maintainability: indicates how easy it is to correct a defect or make a change in the system.
- Portability: relates to the effort it takes to migrate a system or component from one operating environment to another.
- Reusability: indicates the extent to which a component can be used in systems other than the one for which it was initially developed.
- Testability refers to the ease with which the software components or integrated system can be tested to find defects.

Other Important Quality Requirements:

- Performance: refers to the ability of a system to achieve its intended functions within specific time constraints, handling expected workloads, and delivering results without significant delays. It involves how well a system operates in terms of speed, responsiveness, and capacity to process tasks.
- Security: Security refers to protecting a system against unauthorized access, data breaches, and ensuring data integrity and confidentiality.
- Safety: ensures that the system operates without causing harm to people, property, or the environment. It's critical in systems where malfunctions could lead to serious consequences.

Note: Quality requirements must have a **business case** and are not for the sake of the development team. Example: "To improve maintainability, the architecture of the Sales module must be refactored."

Impact of Quality Requirements on Project Success:

- **Product Reliability:** Quality requirements define the level of reliability and robustness expected from the project's output. Failing to meet these requirements can result in product failures, downtime, and increased maintenance costs.

- **User Experience:** Quality requirements also encompass aspects related to the user experience, such as performance, usability, and accessibility. Neglecting these requirements can lead to user frustration and dissatisfaction.

- **Compliance and Standards:** Depending on the industry, there may be regulatory and compliance standards that must be met. Failure to adhere to these requirements can result in legal issues, fines, and reputational damage.

- **Long-term Costs:** Investing in quality upfront may increase short-term project costs, but it often leads to reduced long-term maintenance costs. Neglecting quality can result in higher ongoing expenses and decreased project success.

Constraints: An organizational or technological requirement that restricts the way in which the system shall be developed.

- Constraints restrict the range of alternatives available for realizing the requirements/the entire system.

- Constraints may also lead to the change of requirements or the definition of new requirements

Types of constraints:

- **Cultural** (i.e., constraints originating from the cultural background of the system users).
- **Legal** constraints (i.e., constraints originating from laws and standards).
- **Organizational** constraints.
- **Physical** constraints.
- **Project** constraints.

Or.. it can be categorized with the subject affected by a constraint:

1. **The system itself** (e.g., it may demand or restrict a system function) or
2. **The development process of the system** (e.g., the process model to be used).

Impact of Constraints on Project Success:

- **Budget:** Constraints such as budget limitations can significantly impact a project. Overstepping the budget can lead to financial strain and potential project termination.

- **Time Constraints:** Time constraints, including deadlines and time-to-market pressures, can affect project planning and execution. Failure to meet these constraints can result in missed opportunities or competitive disadvantages.

- **Resource Limitations:** Constraints on resources, including manpower and technology, can impact the project's ability to deliver on functional and quality requirements. Insufficient resources may lead to compromises that affect overall success.

- **Risk Management:** Understanding and managing constraints is essential for effective risk management. Failure to address constraints can increase the project's vulnerability to unexpected challenges and risks.

Problem vs. Solution

"What?" vs. "How?" Different views for different stakeholders, e.g.

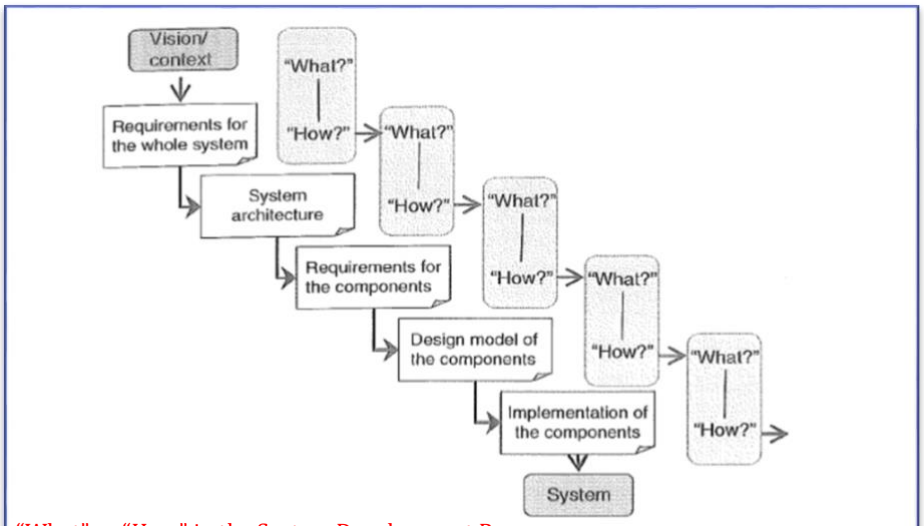
- **Requirements engineering:**

  "What?" – system vision **while** **"How?"** – specified requirements

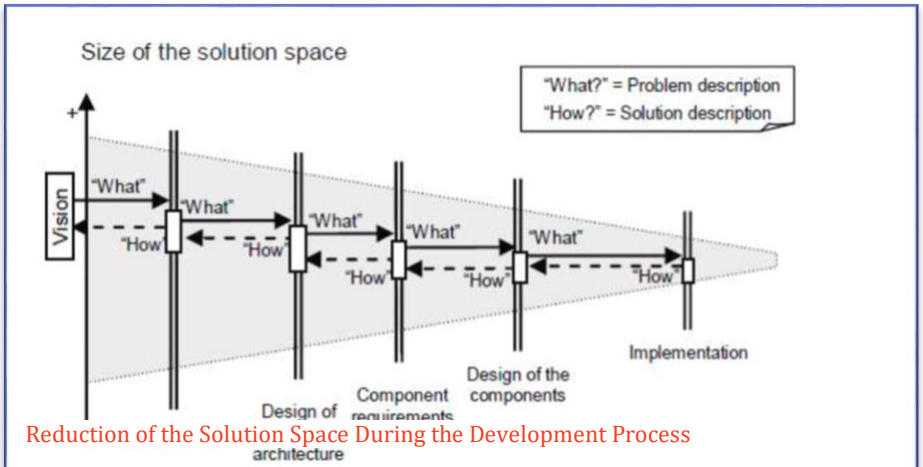
- **System architect:**

  "What?" – system requirements **while** **"How?"** – resulting system

The differentiation between **"what"** and **"how"** also occurs within a single development phase.



"What" vs "How" in the System Development Process



Reduction of the Solution Space During the Development Process

(3) How to Write Good Requirements

Anatomy of a Good Requirement

The Online Banking System shall allow the Internet user **to access** her current account balance **in less than 5 seconds.**

- **Defines the system under discussion.**
- **Modal Verb with correct identifier (shall or may).**
- **Defines a positive end result.**
- **Quality criteria.**

The challenge is to **seek out the system under discussion**, **end result**, and success **measure** in every requirement.

Typical Mistakes in Writing the Requirements

Noise	the presence of text that carries no relevant information to any feature of the problem.
Silence	a feature that is not covered by any text.
Over-specification	text that describes a feature of the solution, rather than the problem
Contradiction	text that defines a single feature in several incompatible ways
Ambiguity	text that can be interpreted in >=2 different ways
Forward reference	text that refers to a feature yet to be defined.
Wishful thinking	text that defines a feature that cannot possibly be validated
Jigsaw puzzles	e.g., distributing requirements across a document and then cross-referencing.
Inconsistent terminology	inventing and then changing terminology.
Putting the onus on the development staff	i.e., making the reader work hard to decipher the intent.

Writing for the hostile reader	(fewer of these exist than friendly ones)

Standards for Writing a Requirement

1. Each requirement must first form a complete sentence (Not a bullet list of buzzwords, list of acronyms, or sound bites on a slide)

2. Each requirement contains a subject and a predicate:
   a. **Subject:** a user type (watch out!) or the system under discussion.
   b. **Predicate:** a condition, action, or intended result.
   c. **Verb in predicate:** "shall" / "will" / "must" to show mandatory nature; "may" / "should" to show optionality.

3. The whole requirement provides the specifics of a desired end goal or result

4. Contains a success criterion or other measurable indication of the quality.

5. Several standards define precisely how to use keywords (verbs and adjectives) in their documents, for example, IETF RFC 2119: *Keywords for use in RFCs to Indicate Requirement Levels:*

   - MUST, REQUIRED, or SHALL: means that the definition is an absolute requirement of the spec.
   - MUST NOT or SHALL NOT: absolute prohibitio.
   - SHOULD or RECOMMENDED: Think twice about not doing it!
   - SHOULD NOT or NOT RECOMMENDED: Think twice about doing it!
   - MAY or OPTIONAL: truly optional.

6. Look for the following characteristics in each requirement:

   - Feasible (not wishful thinking).
   - Needed (provides the specifics of a desired end goal or result).
   - Testable (contains a success criterion/other measurable indication of quality).
   - Clear, unambiguous, precise, one thought.
   - Prioritized.
   - ID.

Note: Several characteristics are mandatory (feasible, needed, testable) whereas others improve communication.

Writing Pitfalls to Avoid

Never describe prematurely how the system is going to achieve something (over-specification), always describe what the system is supposed to do.

1. Refrain from designing the system prematurely. Designing the system too early may possibly increase system costs.
   Danger signs: using names of components, materials, software objects, fields, and records in the user or system requirements.

2. Do not mix different kinds of requirements (e.g., requirements for users, system, and how the system should be designed, tested, or installed)

3. Do not mix different requirements *levels* (e.g., the system and subsystems).
   Danger signs: high-level requirements mixed in with database design, software terms, or very technical terms.

4. Beware: may depend on the level of abstraction… Your ***what*** is my ***how***!

5. Avoid ambiguity and write as clearly and explicitly as possible; ambiguities can be caused by:

   - The word or to create a compound requirement.
   - Poor definition (giving only examples or special cases).
   - The words etc., ...and so on (imprecise definition).
   - Example: "With your meal you have a choice of soup or salad and rice".

6. Do not use vague indefinable terms, many words used informally to indicate quality are too vague to be verified.
   Danger signs: user-friendly, highly versatile, flexible, to the maximum extent, approximately, as much as possible, minimal impact.

7. Do not make multiple requirements. keep each requirement as a single sentence.
   Conjunctions are danger signs: and, or, with, also.
8. Do not ramble.

   - Long sentences with arcane language.
   - References to unreachable documents.

9. Do not speculate.

   - There is no room for "wishlists"–general terms about things that somebody probably wants.
   - Danger signs: vague subject type and generalization words such as usually, generally, often, normally, typically.

10. Do not express suggestions or possibilities.

    - Suggestions that are not explicitly stated as requirements are invariably ignored by developers. **Danger signs:** may, might.

11. Avoid wishful thinking.

    - Wishful thinking means asking for the impossible (e.g.,100%reliable, safe, handle all failures, fully upgradeable, run on all platforms).

(4) RE in Software Development Life Cycles

Predictive, Waterfall-Like Process

- Software development occurred in an orderly series of sequential stages (Progress flows top to bottom, like a waterfall).
- Requirements were agreed to, a design was created, and code followed thereafter. Lastly, the software was tested to verify its conformance to its requirements and design.

The Iron Triangle

- There is a set of **requirements** that can be reasonably determined "up front".
- These requirements can be used as a **basis** to estimate the **schedule** and **budget** of the project.
- This "fixed requirements scope" assumption has indeed been found to be a root cause of project failures.

Iterative and Incremental Process

It emerged as alternatives to the **Waterfall model** due to its **failures** and increasing **time-to-market pressures**.

Boehm's Spiral Process Model

- An initial pass around the spiral is intended primarily to **understand requirements and perform some validation of the requirements** before more serious development begins.
- Thereafter, the model assumed another, larger "spiral" intended to **develop the solution in largely sequential steps of design, coding, integration, and testing.**
- Follows a traditional sequential, waterfall like process, **but incorporates constant feedback.**

The Rational Unified Process (RUP)

- Intended for **large-scale applications** where **robustness**, **scalability**, and **extensibility** are mandatory.
- RUP recognized the **necessary overlap of the various activities.**

- Requirements activities were particularly **intensive** during the **early inception** and **elaboration phases** (as illustrated by the size of the "humps" in the diagram).

- Requirements **elaboration** and requirements **change** are considered to be a **continuous process** that occurs throughout the life cycle.

Agile Process: Principles

- **Highest priority** is to **satisfy the customer** through **early** and **continuous delivery of valuable software.**

- **Welcome changing requirements**, even **late** in development.

- **Deliver working software** (primary measure of progress) **frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

- **Business people** and **developers must work together daily** throughout the project.

- Build projects around **motivated individuals**: Give them the environment and the support they need, and trust them to get the job done.

- In the agile battle of date versus scope, **the date wins.**

- We'll **fix** two things, **schedule**, and **resources**, and we'll **float the scope** (requirements).

Scrum

Scrum is an agile project management method.

- Work is done in "sprints," which are timeboxed iterations of a fixed 30 days or fewer duration.

- Work within a sprint is fixed. Once the scope of a sprint is committed, no additional functionality can be added, except by the development team.

- All work to be done is characterized as **product backlog**, which includes new requirements to be delivered, the defect workload, and infrastructure and design activities.

- A Scrum Master mentors the empowered, self-organizing, and self- accountable teams that are responsible for the delivery of successful outcomes at each sprint.

- A product owner plays the role of the customer proxy.

- A daily stand-up meeting is a primary communication method.

- Typical Scrum guidance calls for fixed 30-day sprints, with approximately 3 sprints per release, thus supporting incremental market releases on a 90- day time frame.

- The **product owner** often **writes the user stories**, with input from the customer, the stakeholders, and the team. In practice, any team member with sufficient domain knowledge can write user stories, but it is up to the product owner to accept and prioritize these potential stories into the product backlog.

User Story

User Story: is a brief statement of intent that describes something the system needs to do for the user.

Distinguishing Use Cases from Software Requirements Specifications (SRS)

- They are **not detailed requirements specifications** but are rather negotiable expressions of interest.

- They are **short**, **easy to read**, and **understandable** to developers, stakeholders, and users.

- They represent **small increments** of valued functionality that **can be developed in a period of days to weeks and can be safely discarded after implementation** (no need for maintenance).

- They are relatively **easy to estimat**e, so effort to implement the functionality can be rapidly determined.

User Story Form

Three elements of a user story:

1. **Card** represents two to three sentences used to describe the intent of the story (details remain to be determined).

2. **Conversation** represents a discussion between the team, customer, product owner, and other stakeholders, which is necessary to determine the more detailed behavior required to implement the intent.

3. **Confirmation** represents the acceptance test, which is how the customer or product owner will confirm that the story has been implemented to their satisfaction.

User Story Voice

It has the following form: ***As a* <role>, *I can* <activity> *so that* <business value>.**

- **<Role>** represents who is performing the action or perhaps who is receiving the value from the activity.
- **<activity>** represents the action to be performed by the system.
- **<business value>** represents the value achieved by the activity.
-

Example: As a consumer, I want to be able to see my daily energy usage so that I can lower my energy costs and usage.

User Story Acceptance Criteria

Acceptance criteria are not functional or unit tests; rather, they are the **conditions of satisfaction being placed on the system.**

Example: *Read DecaWatt meter data every 10 seconds and display on portal in 15-minute increments and display on in-home display every read.*

Epics vs. Themes

A Scrum **epic** is a large user story, while a **theme** is a collection of user stories.

Spikes

- Spikes are a special type of user story (should be put in the backlog) used to drive out risk and uncertainty.
- Spikes may be used for basic research to familiarize the team with a new technology or domain.
- The story may contain significant technical risk, and the team may have to do some research to gain confidence in a technological approach.

- A spike may result in some information, a decision, a prototype, a proof of concept, or some other partial solution, rather than a working code.

- The output of a spike is demonstrable, both to the team and to any other stakeholders.

- Spikes should be the exception, not the rule.

Technical Spikes: Used to research various technical approaches in the solution domain. For example, a technical spike may be used to determine a build-versus-buy decision, to evaluate potential performance of a new user story, to evaluate specific implementation technologies.

Functional spikes: Used whenever there is significant uncertainty as to how a user might interact with the system. Functional spikes are often best evaluated through some level of prototyping, whether it be user interface mock-ups, page flows, etc.

A spike Example: As a consumer, I want to see my daily energy use in a histogram so that I can quickly understand my past, current, and projected energy consumption.

In this case, a team might create two spikes:

1. **Technical spike:** Research how long it takes to update a customer display to current usage, determining communication requirements, bandwidth, and whether to push or pull the data.

2. **Functional spike**: Prototype a histogram in the web portal and get some user feedback on presentation size, style, and charting attributes.

Return on Investment (ROI)

In **waterfall**, investment (cost) starts immediately and continues until delivery is reached.

In **agile**, value delivery starts with the first shippable increment.
The sooner we deliver a feature, the sooner our customers will pay us for it

ROI $$ (agile) > ROI $ (waterfall)

(5) System Vision, Context and RE Framework

Vision

- The Vision is an **outcome of the company's business** and **portfolio investment strategy**.
- Vision states a **goal**, not how to achieve it.
- Vision is guidance throughout the development process.
- **The system vision is a joint vision of the system agreed upon by all active stakeholders.**
- **Vision characteristics:** Big picture and Abstract.
- **Vision purpose:** Agreement on what this project is about and Easy communication with stakeholders.

Example of Vision Statement Template

1. **For** [target customer]
2. **Who** [statement of the need or opportunity]
3. **The** [product name]
4. **Is** [a product category]
5. **That** [key benefit, compelling reason to buy or use]
6. **Unlike** [primary competitive alternative, current system, or current business process],
7. **Our product** [statement of primary differentiation and advantages of new product]

Vision in Context

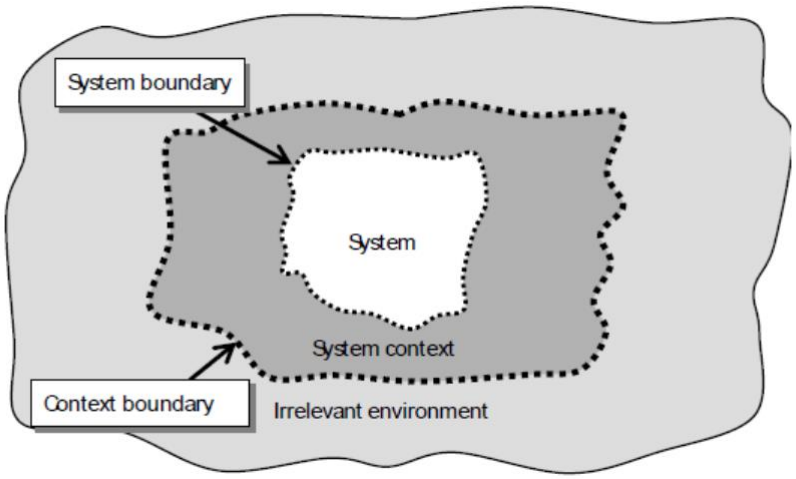
The main goal of RE is to: **"establish a vision within an existing context".**

System Context, System Boundary, Context Boundary

- The **system context** is the part of the system environment that is relevant for the definition as well as the understanding of the requirements of a system to be developed.

- The **system boundary** separates the system to be developed from its environment, i.e., it separates the part of the reality that can be modified or altered by the development process from aspects of the environment that cannot be changed or modified by the development process.

- The **context boundary** separates the relevant part of the environment of a system to be developed from the irrelevant part, i.e., the part that does not have to be considered during requirements engineering.



Grey Zone between the System and the Context: During requirements engineering, the system boundary as well as the interfaces are **typically unstable**, i.e., **they change quite frequently**.

Hints About System Boundary

- Determine explicitly which aspects **belong to the system**.

- Determine which aspects are **outside the system boundary**.

- When defining the system boundary **involve all relevant stakeholders**.

- Try to reach an agreement about the system boundary. In case you cannot decide whether some object belongs to the system or not, put it in the **grey zone**.

- **Check periodically** whether the defined system boundary is still valid. Pay attention to required extensions or reductions of the system boundary.
- If the system boundary needs to be **adjusted**, **verify whether the adjustment impacts the already defined requirements.**

Hints About Defining the Context Boundary

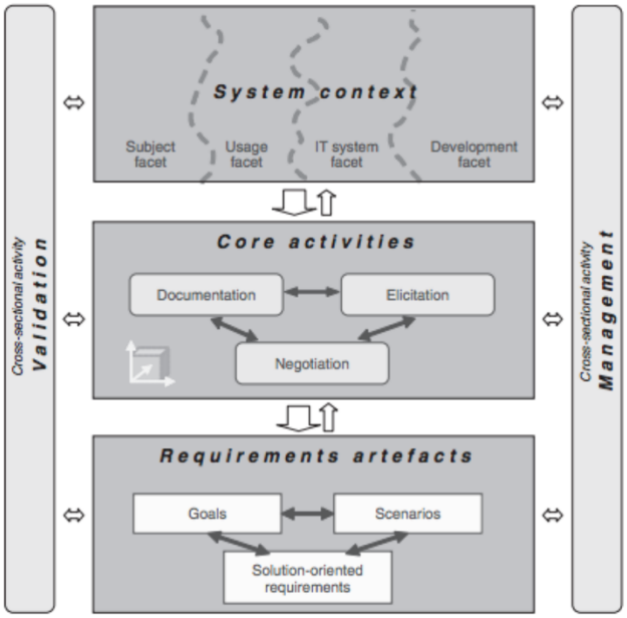
- If you are unsure whether some context aspect impacts the system requirements or not, assign it to the gray zone.

- When you conclude that one context aspect is irrelevant to the system, **document this aspect** as part of the **irrelevant environment**.

- When defining **new requirements** (e.g., functions) check whether context aspects (e.g., a law) classified as irrelevant so far become relevant due to the new requirements.

- **Use goals and scenarios to check whether specific aspects of the environment are relevant to the system or not. If an aspect is relevant, it should affect at least one goal or one scenario.**

The Requirement Engineering Framework



(1) System Context: Four Facets

1. **Subject facet**: Objects and events relevant to the system.
   Example: Elements the system must store or process information about.

2. **Usage facet**: Aspects concerning the usage by people or other systems.
   Example: Different user groups with specific characteristics, laws, and standards restricting or influencing the system usage.

3. **IT system facet**: Objects and elements of the IT system environment of the system.
   Example: Existing hardware and software components to be used.

4. **Development facet**: Aspects concerning the development process of the system.
   Example: process guidelines, development tools.

(2) Core Activities

1. **Elicitation** (Content Dimension)
   - Requirements are elicited from stakeholders and other sources.
   - New and innovative requirements are collaboratively developed.
   - The goal is to improve the understanding of the requirements.

2. **Negotiation** (Agreement Dimension)
   - All conflicts between the viewpoints of the different stakeholders have to be detected and made explicit.
   - The identified conflicts should be resolved.

3. **Documentation** (Documentation Dimension)
   Documentation and specification of the elicited requirements according to the defined documentation and specification rules.

(3) Requirements Artefacts

The three following requirements artifacts types (Goals, Scenarios, and solution-oriented) are used complementarily.

1. **Goals**
   - A goal is an **intention** with regard to the objectives, properties, or use of the system.
   - A goal **should be solution-free.** Stakeholders typically have many different alternatives for satisfying a goal.

2. **Scenarios**
   - A scenario describes a concrete example of **satisfying** or **failing to satisfy** a goal (or a set of goals). A scenario typically defines a **sequence of interaction steps** executed to **satisfy the goal** and relates these interaction steps to the system context.

3. **Solution-oriented requirements**
   - Define the data perspective (entities, attributes, relationships), the functional perspective, and the behavioral perspective on a software-intensive system.

   - In contrast to goals and scenarios, which should be defined **fairly independently from a specific and intended solution**, **the definition of solution-oriented requirements often implies a conceptual (or logical) solution for the system.**

(4) Two Cross Sectional Activities

1. **Validation**
   - Validation of the requirements artifacts, core activities, and consideration of the system context

2. **Management**
   - Management of artifacts and activities.

   - Observation of the system context.

   - Activities are interrelated, for example performing one activity may require the execution of additional activities.

(6) Requirements Discovery

Requirement Elicitation

Requirements elicitation is "the process of **discovering** the requirements for a system by communicating with customers, system users, and others who have a stake in the system development".

Sources of Requirements during Elicitation

1. **Various stakeholders (**Clients, customers, users (past and future), buyers, managers, domain experts, developers, marketing and QA people, lawyers, people involved in related systems, and anyone who can bring added value!).

2. Pre-existing systems (Not necessarily software systems, e.g., manual workflows).

3. Pre-existing documentatio.

4. Competing systems.

5. Documentation about interfacing systems.

6. Private data (companies) and public data (social media).

7. Standards, policies, collective agreements, legislation.

Stakeholders

- **Customer/ Client**
  - Person who pays for the software development.
  - Ultimately, has the final word on what the product will be.

- **User** of the current system or future system
  - Experts of the current system: indicate which functions to maintain or improve.
  - Experts of competitors' products: suggestions on designing a superior product.
  - May have special needs or requirements (Usability, training, online help ...)
  - Do not neglect interest groups (Expert users, users with disabilities or handicaps)
  - Select users with care (Different seniority and other categories, Must speak with authority and be responsible and motivated)

- **Domain Expert**
  - Expert who knows the work involved.
  - Familiar with the problem that the software must solve.
  - Also knows the environment in which the product will be used.

- **Software Engineer**
  - Expert who knows the technology and process.
    Determines if the project is technically and economically feasible.
  - Specifically estimates the cost and time of product development.
  - Educates the buyer/client on the latest and innovative hardware or software, and recommends new features that will benefit from these technologies.

- **Inspector**
  - An expert in governmental rules and safety relevant to the project.

- **Market research specialist**
  - Can play the role of the customer if the software is developed for the general public
  - Expert who has studied the market to determine trends and potential needs of (future) customers.

  - **Lawyer**
    Familiar with laws and legal/contractual aspects.
  - Standards relevant to the project.

- **Expert on interacting systems**
  - Knows the interfaces of the (external) interacting systems.
  - May be interested in product features (if the product can help he interacting system perform its tasks).

- **Others that bring added value**
  - People who will use your product as a basic building block.
  - User experience (UX) expert, negative stakeholders (competitor, hacker, ...), regulators, unions, testers, interest groups, the "crowd", etc.

- **Managers and decision-makers**

Elicitation Risks and Challenges

1. **Problems of scope**
   - System boundaries inadequately defined or defined too soon.
   - Unnecessary technical details.

2. **Problems of understanding**

- Stakeholder not sure of what is needed.
- Stakeholder has trouble communicating needs
- Stakeholder does not understand the capabilities and limitations of the computing environment. Requirements are limited by what stakeholders *think* is possible.
- Stakeholder does not have a full understanding of the domain.
- Stakeholders state conflicting requirements Hard to detect, negotiate, and prioritize.

3. **Problems of volatility**
   - Stakeholders will not commit to a set of written requirements.

4. **Other typical issues**
   - Experts seldom available.
   - Many stakeholders lack motivation or resist to change.
   - Finding an adequate level of precision/detail.
   - Mixing requirements with design.
   - Common vocabulary often missing.

5. **Requirements do not fall from the sky!**
   - Sometimes hidden.
   - Sometimes too obvious, implicit, ordinary...

Persona

- Fictitious representation of a probable user (Away from outlier and unlikely cases).
- Seeks to represent the needs and characteristics of different groups of users (often unavailable).
- Ideally created from real data/encounters/observations.
- Can partially compensate for the physical absence of users.
  - Name and photo! Personalizes a particular user. *Focus* and *empathy*!
  - No interactions, however ...
- Describes: behavior patterns, objectives, problems, abilities, attitudes, environment, culture, demographic info, ...
- Different templates exist. Some are generic and some focus on particular areas.

Characteristics of personas: VARIED

- **Vivid:** The persona should make anyone who reads it feel like they have actually met this person.
- **Actionable:** If the persona does not inform how you sell or build stuff, why bother?
- **Real:** Good personas are not created in cubicles. Go where the persona is and observe!
- **Identifiable:** Make sure you can identify and target these personas, or you will not be able to find a use for them.
- **Exact:** "Everyone" is not your customer. Make sure the personas are distinct so you can apply relevant focus.
- **Detailed:** People are complicated so good personas are usually substantial.

Requirements Elicitation Techniques

(1) The Requirements Workshop (Focus group)

- Key stakeholders of the project are gathered together for a short, intensive period, typically no more than a day or two.
- **Goal:** Drive **consensus** on the requirements of the system or application and gain rapid **agreement** on a course of action from the key stakeholders, in a **very short time**.
- The workshop is based on the **brainstorming process.**
- The workshop may be facilitated by a **product owner**, **product manager**, **team member**, or **outside facilitator**.

Workshop Challenges

These decision-making workshops are often characterized by **a highly charged atmosphere (excitement or nervousness).**

Difficult to get consensus, including different **opinions** and **expectations** for the solution requirements and the **impact on resources and budgets.**

Brainstorming Process

Brainstorming does the following:

- Encourages stakeholders to **contribute**.
- Has high bandwidth—many ideas/potential solutions can be generated in a short period.
- Encourages out-of-the-box thinking—unlimited by the usual constraints.

Brainstorming has two main phases:

1. Idea generation 2. Idea reduction **Optional step**: Idea Prioritization.

Interviews and Questionnaires

- A **simple** and **direct** technique that can be used in virtually every situation.
- **As solution providers, we rarely find ourselves in a situation in which we have no idea what types of potential solutions would address the problem.**
- However, we must not let our current context interfere with a better understanding of a new problem to be solved.

(2) Interviews

Interview - a fact-finding technique whereby the systems analysts collect information from **individuals** through **face-to-face interaction.**

- Find facts.
- Verify facts.
- Clarify facts.
- Get the end-user involved.
- Identify requirements.
- Solicit ideas and opinions.

The personal interview is generally recognized as the most important and most often used fact-finding technique.

Types of interviews:

- **Unstructured interview** –conducted with only a **general goal** or subject in mind and with **few**, if any, **specific questions**. The interviewer **counts on the interviewee to provide a framework and direct the conversation.**

- **Structured interview** –the interviewer has a specific set of questions to ask the interviewee.

Types of questions:

- **Open-ended question** – a question that allows the interviewee to respond in any way.

- **Closed-ended question** – a question that restricts answers to either specific choices or short, direct responses.

Interviews Advantages and Disadvantages

Advantages:

- Give analyst the opportunity to motivate interviewee to respond freely and openly.
- Allow analyst to probe for more feedback.
- Permit analyst to adapt or reword questions for each individual.
- Can observe nonverbal communication.

Disadvantages:

- Time-consuming.
- Success highly dependent on analyst's human relations skills.
- May be impractical due to location of interviewees.

Procedure to Conduct an Interview

1. Select Interviewees (stakeholders, e.g., end-users). Learn about the individual prior to the interview.

2. Prepare for the Interview. Follow the interview guide.

3. Conduct the Interview.

4. Follow Up on the Interview. Memo that summarizes the interview

Do

- Dress appropriately.
- Be courteous.
- Listen carefully.
- Maintain control of the interview.
- Probe.
- Observe mannerisms and nonverbal communication.
- Be patient.
- Keep the interviewee at ease.
- Maintain self-control.
- Finish on time.
- Understand the background of the stakeholder to be interviewed.

Don't

- Assume anything about the topic or the interviewee.
- Assume an answer is finished or leading nowhere.
- Use complex questions.
- Reveal personal biases.
- Ask threatening questions, e.g., "Do you think that your manager knows about the system security issues and ignored them?"
- Talk more than listen.
- Tape record (take notes instead).
- Ask Loaded/Biased questions, e.g., "Do you treat your children with kindness like a good parent should?"

(3) Questionnaires

Questionnaire – a special-purpose document that allows the analyst to collect information and opinions from respondents. Also, **referred to as Surveys.**

Free-format questions – a question designed to offer the respondent greater latitude in the answer. A question is asked, and the respondent records the answer in the space provided after the question.

Fixed-format questionnaire – a question that requires selecting an answer from predefined available responses. (Ranking, Rating, and MCQ Questions).

Questioners Advantages and Disadvantages

Advantages:

- Used when it is impossible because of time, distance or cost constraint, to interview all stakeholders.
- Often can be answered quickly.
- People can complete at their convenience.
- Relatively inexpensive way to gather data from a large number.
- Allow for anonymity.
- Responses can be tabulated quickly.

Disadvantages:

- Return rate is often low.
- No guarantee that an individual will answer all questions.
- No opportunity to reword or explain misunderstood questions.
- Cannot observe body language.
- Difficult to prepare.

Developing a Questionnaire

1. Determine what facts and opinions must be collected and from whom you should get them.

2. Based on the facts and opinions sought, determine whether free- or fixed-format questions will produce the best answers.

3. Write the questions.

4. Test the questions on a small sample of respondents.

5. Duplicate and distribute the questionnaire.

Good Questionnaires Design

- ✓ Begin with non-threatening and interesting questions.
- ✓ Group items **into logically coherent sections.**
- ✓ Do not put important items at the very end of the questionnaire.

- Do not crowd a page with too many items.
- Avoid abbreviations.
- Avoid biased or suggestive items or terms.
- Number questions to avoid confusion.
- Provide secrecy to respondents.

A Note on Questionnaires

Although questionnaires are often used and appear scientific because of the opportunity for statistical analysis, a questionnaire is **not a substitute for interviewing** in the requirements discovery context because it has some fundamental problems.

- **Relevant questions** cannot all be decided in advance.
- The assumptions behind the questions bias the answers.
- It is difficult to explore new domains ("What you really should be asking about is ...").
- It is difficult to follow up on ambiguous user responses.
- There is no substitute for the personal contact, rapport building, and free-form interaction of the interview.
- Do the interview first and do it for every new class of problem.

Typical Errors in Survey Questions

- *How dumb is Donald Trump when it comes to foreign policy?*
  Leading question, which sways the reader to a particular side.
  Better: *Please describe Donald Trump's position on foreign policy.*

- *Where do you like to drink coffee?*
  Loaded question: what if one does not drink coffee or caffeinated drinks? That person cannot answer this question truthfully.
  Better: use skip logic *Do you drink Coffee? If not, please skip the next question Where do you like to drink coffee?*

- *How happy or unhappy are you with the tuition fees and with your course curriculum?*
  Double-barreled question: these are two different things...
  Better: *How happy or unhappy are you with the tuition fees?* Also: *How happy or unhappy are you with your course curriculum?*
- *Do you always shower before bed?*
  Absolute question: participant is cornered and the answer will always be no...
  Better: *How many nights a week do you shower before bed?*

- *How many nights a week do you shower before bed: Every day, 5-6 days, 3-4 days, 1-3 days, 0-1 days?*
  Overlapping answers: keep the multiple choice answers mutually exclusive
  Better: *6-7 days, 4-5 days, 2-3 days, 0-1 day?* Also, as this might be a sensitive topic, one option could be "prefer not to answer"

- *What was the quality of the service you received: Fair, Good, Very Good, or Excellent?*
  Unbalanced scale: No option related to Poor service.

- *What type of vehicle do you own: Van, Sedan, or SUV?*
  Incomplete scale: there exist many other options. Minimally, the options should include Other. One may not own a vehicleeither...

(4) Analysis of Existing System

Useful when building a new improved version of an existing system

Why analyze an existing system?

- Users may not like the new system if it is too different or does not do what they want (risk of nostalgia for old system).

- To take into account real usage patterns, human issues, common activities, relative importance of tasks/features.

- To catch obvious possible improvements (features that are missing or do not currently work well).

- To find out which "legacy" features can/cannot be left out.

Review Available Documentation

- Start with reading available documentation User documents (manual, guides...).
- Development documents.
- Requirements documents.
- Internal memos.
- Change histories.
- …
- Of course, often these are out of date, poorly written, wrong, etc., but it's a good starting point.

Ethnography

- **Ethnography is an observational technique that can be used to understand operational processes and help derive support requirements for these processes.**

- Essentially seeks to **explore the human factors** and **social organization of activities** → **understand the work.**

- **Useful to discover for example:**
  - What does a nuclear technician do during the day?
  - What does his workspace look like?

- **Less useful to explore political factors:**
  - Workers are aware of the presence of an outside observer.

- Discoveries are made by **observation** and **analysis** (**initially workers are not asked to explain what they do**)
  - Collect what is ordinary/what people do.
  - Study the context of work and watch work being done.

- Next, ask the user to explain everything he/she is doing .

- Session videotaping.

- Can be supplemented later with interviews/questionnaires.

User Experience Mock-Ups

- **"A picture is worth a thousand words".**

- Developing **user experience interfaces** is always a challenge.

- All aspects of the **presentations**, **dialogues**, and **flows** that make a user interface **easy to navigate** are likely to be largely **unexplored** prior to beginning implementation→**resulting in real frustration**.

- An easy-to-change **drawing** or **mock-up** is an early and effective means of communicating elements of user presentation.

Competitive Analysis

- Who is our competitors, and what features are they offering?
- A competitive analysis is usually presented as a matrix.
- Hire market survey companies that will research a domain and produce reports outlining prospective benefits of a product proposition.
- Most companies do their own competitive analysis; that way, they gain the knowledge directly.
- In its simplest form, a **competitive analysis** consists of the following:
  - Refining the domain of interest/product category.
  - Identifying competitors.
  - Studying competitive offerings.
  - Preparing an analysis of the finding for evaluation.

Customer Change Request Systems

- Many companies maintain a customer-facing enhancement request system for just this purpose.
- Customers provide feedback on our roadmaps and generate new concepts for products and features.
- Product managers pull ideas from the community into their life-cycle management system.
- Integrate concepts directly into their roadmap—**without having to run traditional focus groups, use off-line surveys, or wait for an annual user conference. It all happens 24×7.**

- Developers can then more efficiently develop the right features, already vetted by customers.
-
- Customers can participate through the entire process—they track the features they care about, and are notified when they are released.

Defect Logs

- Provide a **rich source of product requirements**, because **typically many of the reported "defects" are really enhancement requests**.

- A high number of enhancement requests for a particular capability item is a relatively unbiased form of prioritization (at least from the perspective of *existing* customers).

Use Case Modeling

When it comes to systems of complexity—systems that are composed of other systems, systems that contain hardware devices and software components, and suites of applications that work together to provide even higher value to the user—neither features nor user stories are sufficiently capable constructs to describe this complex, *aggregate* behavior.

For these systems, use cases describes the interplay among the actors (users, devices, subsystems) and the various systems that work together to deliver this behavior.

(7) Fundamentals of Scenarios

Scenarios

What is a Scenario ?

- **Goals** alone **do not sufficiently support requirements elicitation.**
- Stakeholders typically find it easier to communicate their requirements in terms of examples (e.g., **interaction sequences**) rather than **abstract intentions.**
- A **scenario** describes a **concrete example** of **satisfying** or **failing** to satisfy a **goal** (or set of goals).
- A scenario typically defines **a sequence of interaction steps** executed to satisfy the goal and relates these interaction steps to the system context.

Scenarios as a Means for Putting Requirements in Context

Scenarios are well suited for documenting **context information**. Kinds of context information:

1. **Actors**: Persons or systems interacting with the system.
2. **Roles**: Specific class of actors.
3. **Goals**: Scenarios illustrate satisfaction of goals.
4. **Precondition**: define conditions that must hold before executing the scenario.
5. **Post conditions**: must hold within the system or context after executing the scenario.
6. **Resources**: special preconditions referring to persons, information, financial or other material resources needed for a scenario.

Positive and Negative Scenarios

- Scenario can be **positive** or **negative** (regarding the satisfaction of a goal).
- **Positive** scenario documents sequence of interactions leading to **the satisfaction of a goal.**
- **Negative** scenario documents a sequence of interactions **failing to satisfy a goal** . A Negative scenario may be **allowed** or **forbidden.**
- **Positive and negative scenarios complement each other.**

Forbidden Negative Scenarios (failure scenario)

- The failure to satisfy the related goals **cannot be tolerated.**
- For instance, because the respective **goals are critical** for the system and/or its context, the execution of the sequence is regarded as a **system failure**.
- The stakeholders must take appropriate measures to **prevent the execution of the sequence of interactions documented in a forbidden negative scenario.**

Descriptive and Exploratory/Explanatory Scenarios

1. Descriptive scenario: Describes **process** or **workflow.**

Purpose: understanding its operations.

2. Exploratory/Explanatory scenario: Explore and evaluate possible alternative solutions in order to support the selection of **one alternative solution**.

Provides background information and rationales for particular interaction sequences.

Instance and Type Scenarios

1. **Instance scenario:** Describes a **concrete** (existing or envisioned) **sequence of interactions between concrete actors**
2. **Type scenario:** **Abstracts** from the concrete actors, inputs and outputs of a specific sequence of interactions
3. **Mixed scenario**
   - Important content described at **instance level.**
   - **Content not completely understood** is described at **instance level** to **avoid errors resulting from early abstractions.**
   - Content that is **well understood** described at the **type level.**
   - **Conflicting** or **potentially conflicting content** described at **instance level.**

System-Internal, Interaction, and Context Scenarios

1. **System-internal (type A) scenarios:** Focus **exclusively** on **system-internal interactions** (interactions that occur within the system boundaries).

2. **Interaction (type B) scenarios:** Document sequences of **interactions between a system and its actors** (persons and systems in the context).

3. **Context (type C) scenarios:** Document additional context information relevant to the system usage

Primary versus Secondary Actors

1. **Primary Actors:** The Actor(s) using the system to **achieve a goal**. The Use Case **documents the interactions between the system and the actors to achieve the goal of the primary actor.**

2. **Secondary Actors**: Actors that the system **needs assistance from** to achieve the primary actor's goal.

- **Primary actor initiates the use case.**

- Secondary actors (also called **supporting actors**) **may or may not have goals** that they expect to be satisfied by the use case, the primary actor **always has a goal**, and the use case exists to satisfy the primary actor.

- Secondary actors are **required for the successful completion** of the Use Case.

Use Case Diagram (UCD)

- Built in early stages of development.

- **Purpose**:
  - Model the **context of a system**
  - Capture the requirements of a system

- Diagram that shows **a set of use cases and actors and their relationships.**

- Actors may be connected to use cases only by **association**.

- An association between an actor and a use case indicates that the actor and the use case **communicate with one another**, each one possibly sending and receiving messages.

- However, they do not capture the full information of the actual use cases →**textual description is essential.**

UML: What is an Actor?

The Unified Modeling Language (UML) defines an Actor as: **"An actor specifies a role played by a user or any other system that interacts with the subject"**

- Actors **are NOT part of the system.**

- Include system components only if they are responsible for initiating/triggering a use case.

- Include all **user roles** that interact with the system.

- Actors are **not individual persons** (e.g., Omar) but stimulate the system to react (primary actor).

- **In UML, an actor is represented as a stickman.**

What is a use case?

- A **case of use** of the system/product.

- **Tells a story:** A use case is a **description** of **a sequence of events** involving **interactions of a user with the system.**

- A use case describes **what a system does** but does not specify how it does it.

- A use case typically represents a major **functionality** that is **complete** from beginning to end.

- **Is oriented toward satisfying a user's goal.**

Basic elements of use case diagram

Actor: is someone interacting with a use case (system function). Named by a **noun**.

Use Case: specifies system function (process – automated or manual). Named by a **verb** (Do something).
Each Actor **must be linked to a use case, while some use cases may not be linked to actors.**

How do we describe use cases?

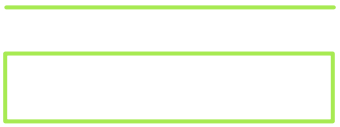
- Textual or tabular descriptions.
- Use Case Diagrams .

In the UML, a use case is represented as an oval.

Identifying Use Cases

- **What will the actor use the system for**?
  - Describe the functions that the user will want from the system
- **Will the actor create, store, change, remove, or read data in the system**?
  - Describe the operations that create, read, update, and delete nformation
- **Will the actor need to inform the system about external events and vice versa**?
  - Describe how actors communicate information about events that the system must know about
  - Describe how actors are notified of changes to the internal state of the system

Elements of a Use Case Diagram



Connection between Actor and Use Case.

Boundary of the system.

Include relationship between Use cases.

Extend relationship between Use cases.

Include Relationship

- The base use case explicitly incorporates the behavior of another use case at a location specified in the base.

- **The included use case never stands alone**. It only occurs as a part of some larger base that includes it.

- **Reuse:** Enables us to **avoid describing the same flow of events several times by putting the common behavior in a use case of its own**.

- A standard case linked to a **mandatory** use case. **Example**: to Authorize a Car Loan (standard use case), a clerk must run Check Client's Credit History (include use case).

- Standard use case can NOT execute without the include case →**tight coupling**.

- The standard UC **includes** the mandatory UC (use the verb to figure direction arrow).

Extend Relationship

1. **Extended** use case (i.e., base) is meaningful on its own and is **independent of the extending use case.**
2. **Extending** use case typically defines optional behavior that is **not** necessarily **meaningful by itself.**
3. The extension takes place at one or more extension points defined in the extended use case.
4. Extend relationship is shown as a dashed line with an open arrowhead directed from the extending use case to the extended (base) use case. The arrow is labeled with the keyword «extend».
5. The new functionality may open up a whole raft of possibilities, and **there is a danger that the Alternative Flow spawns further sub-flows.**
6. The Use Case may become **difficult to manage**. To avoid this, the «extend» relationship can be used to pull the Alternative Flow and its sub-flows out into a new Use Case.
7. The «extend» relationship says that we execute the base Use Case, but when we get to a specified point in the flow, if the right conditions are met, we perform some different steps.

Basic and Alternative Flows

Basic flow: Flow that represents the most common path (the "happy path") from start to end through the use case.

- What actor's event starts the use case?
- How does the use case end?

A number of alternate flows based on both regular circumstances and exceptional events.

The following questions can help discover these paths:

- How does the use case repeat some behavior?
- What else can the actor do?
- How will the actor react to optional situations?
- What exceptions to the usual behavior may occur?

Generalization in UCD

Actor Generalization:

- There is **duplicate behavior** in both, e.g., the buyer and seller which includes "create an account" and "search listings".
- Extract a **more general user** that has the **duplicate behavior** and then the actors will "**inherit**" this behavior from the new user.

UC Generalization:

- The child use case **inherits** the behavior and meaning of the parent use case.
- The child may add to or override the behavior of its parent.

Misuse Scenarios/Cases

- Actor is a **hostile agent,** called also **mis-actor or misuser.**
- A "misuse case" describes a **sequence of interactions in which a hostile actor uses the system against the stakeholders' intentions.**
- The execution of a misuse scenario represents **a threat for the system**, the **stakeholders, or other systems in the context.**
- Misuse cases naturally focus on **NFRs**, e.g., **Safety**, **Security.**
- Valuable for **hazard** and **threat analysis.**
- Misuse cases are **negative use cases.**

Two new relations:

- ***Mitigates:*** A use case can mitigate the chance that a misuse case will complete successfully.

- ***Threatens:*** A misuse case can threaten a use case, e.g., by exploiting or hindering it to achieve its goals.

Misuse Cases Documentation

There are two ways to textually describe a misuse case:
1. Embedded in a use case description template - where you add an extra description field called **Threats**. This is the field where you fill in your misuse case steps (and alternative flows). **This is referred to as the lightweight mode of describing a misuse case.**

2. Use of a separate template. Inherit some of the fields from the use case description (**Name**, **Summary**, **Author**, and **Date**). It also adapts the fields: **Basic path** and **Alternative path**, where they now describe the paths of the misuse cases instead of the use cases.

UML Activity Diagram

- Model the flow of activity/events **from a start point** to **the finish point** detailing the many **decision paths** that exist in the progression of activities/events contained in the activity.

- May be used to detail situations where the **logic is complex** and there are **a lot of alternate flows** (e.g., **parallel processing** may occur in the execution of some activities).

- Typically used for **business process modeling**, for modeling the logic captured by a single use case or usage scenario, or for modeling the detailed logic of a business rule.

- If customers prefer diagrams over text.

Activity: An activity is shown as a round-cornered rectangle **enclosing** all the **actions**, **control flows** and **other elements** that make up the activity.

Actions: An action represents a **single step** within an activity (**one that is not further decomposed within the activity**).

- In Enterprise Architect (EA), it is referred to as "**Atomic**" action.
- Actions are denoted by **round-cornered rectangles**.
- **Constraints** can be attached to an action (e.g., **local pre- and post-conditions**).

Initial Node An initial or start node is depicted by a large black spot. (**You can have more than one initial node).**

Activity Final Node: Denotes the end of all control flows within the activity, and depicted as a circle with a dot inside.

- An activity may have **more than one activity final node**. The first one reached **stops all flows** in the activity.

Objects: An object is shown as a rectangle.

Control Flow: A control flow shows the flow of control from one action to the next. Its notation is a line with an arrowhead, it may have a condition attached to it.

Object Flows

- An object flow is a path along which objects or data can pass.
- An object flow is shown as a connector with an arrowhead
- denoting the direction the object is being passed.

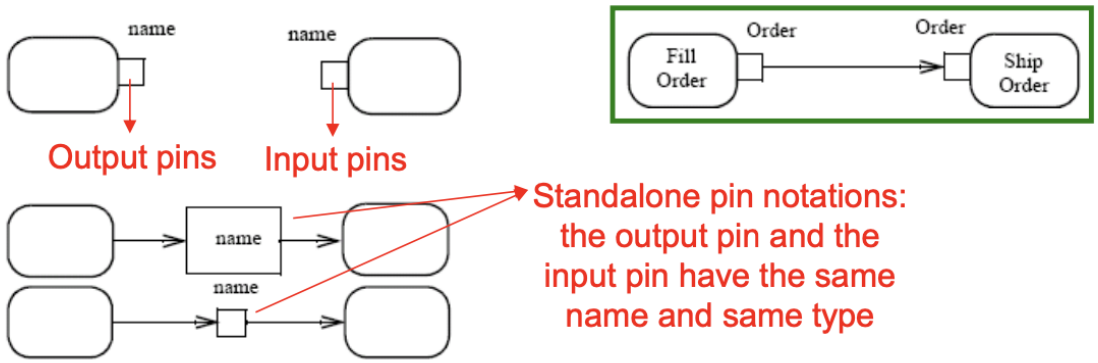
Flow Final Node

- Depicted as a circle with a cross inside.
- Denotes the end of a **single control flow.**
- A flow final destroys all tokens that arrive at it. **It has no effect on other flows in the activity.**
- **You can have more than one flow final node.**

Pins

- Actions can have **inputs** and **outputs**, through the pins.
- Hold inputs to actions until the action starts, and hold the outputs of actions before the values move downstream.

- The name of a pin is not restricted: generally recalls the type of objects or data that flow through the pin.



Decision and Merge Nodes : a **diamond** shape notatio.

- They can both be named.
- The control flows coming away from a decision node will have guard conditions which will allow control to flow if the guard condition is met.

Partitions

- An activity partition is shown as either **a horizontal or vertical swimlane**.
- The partitions are used to **separate actions** within an activity into those performed by the accounting department and those performed by the customer.

Fork and Join Nodes

- Forks and joins have the same notation: either a horizontal or vertical bar. (the orientation is dependent on whether the control flow is running left to right or top to bottom).
- They indicate the start and end of concurrent threads of control.
- A join node may have two or more incoming legs. For continuation it's necessary that all reach the join node.

Join Specification Feature: If only some of the arriving tokens shall be sufficient to continue with the synchronized path, UML provides the *Join Specification* (JoinSpec) feature. By this you may specify a condition, sufficient for synchronization.

Merge vs. Join Nodes

- A join is different from a merge in that the join synchronizes two inflows and produces a single outflow.
- The outflow from a join cannot execute until all inflows have been received.
- A merge passes any control flows straight through it.
- If two or more inflows are received by a merge symbol, the action pointed to by its outflow is executed two or more times.

(8) Solution Oriented Requirements

UML Diagrams Most Relevant for R.E.

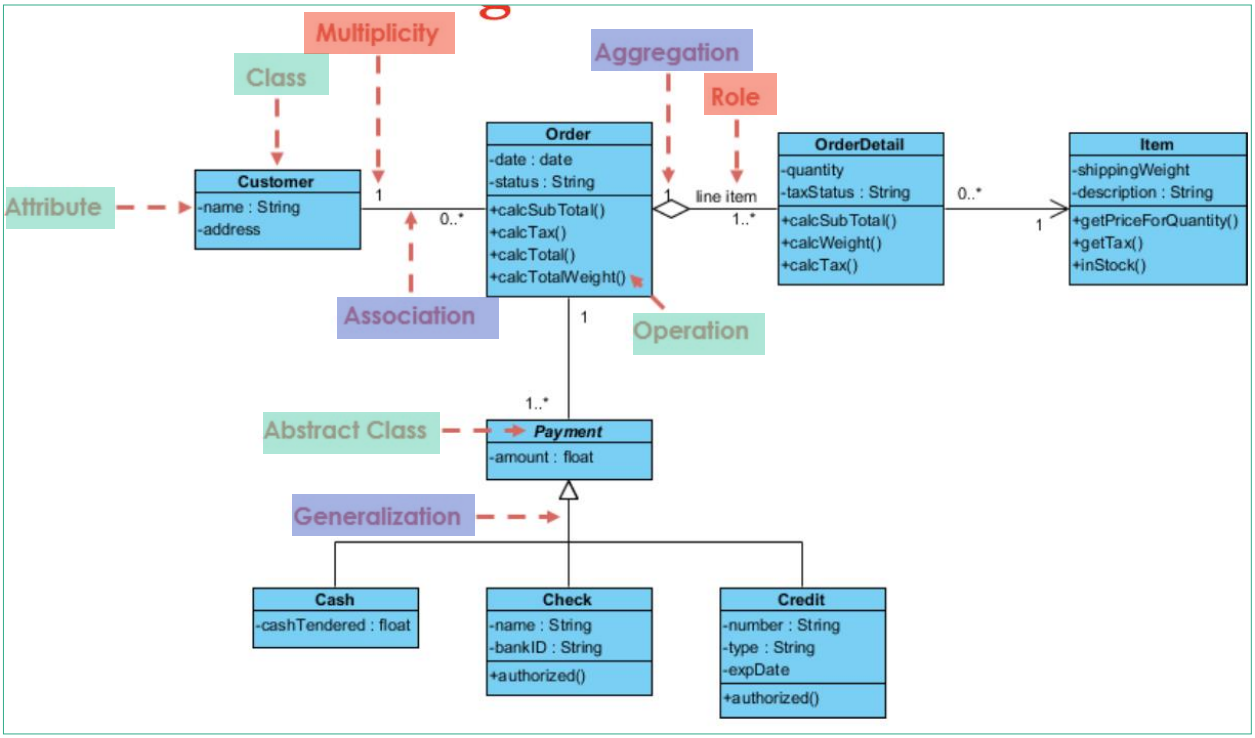
- ***Use case diagram***
  Use cases structuring.

- ***Activity diagram***
  Workflow and process modeling.

- ***Class diagram***
  Domain modeling.

- ***State machine diagram***
  - Detailed behavioral specification (of objects, protocols, ports...).
  - System behaviour (black box).
  - Object/document lifecycles.

UML Class Diagram



Summary of Relationships:

1. ***Association***
   - A general "has-a" or "uses-a" relationship between two classes.
   - Example: Student — enrolls in — Course

2. ***Aggregation*** (◇)
   - A **whole-part** relationship where the part **can exist independently** of the whole.
   - Example: Team ◇— Player (players still exist if team is disbanded)

3. ***Composition*** (◆)
   - A **stronger whole-part** relationship where the part **cannot exist without** the whole.
   - Example: House ◆— Room (a room cannot exist without a house)

4. ***Generalization / Inheritance*** (△)
   - An "is-a" relationship. A subclass inherits from a superclass.
   - Example: Dog ⬆ Animal (Dog is an Animal)

Multiplicity Notations:

Symbol	Meaning	Example
1	Exactly one	A **Passport** must be assigned to **exactly one Person**
0..1	Zero or one	A **Person** may have **zero or one Driver's License**
*****	Zero or more	A **Customer** can place **zero or more Orders**
1..*	One or more	A **Company** must have **at least one Employee**
n..m	Between n and m inclusive	A **Team** must have **between 2 and 5 Players**

Domain Modeling in RE

It's important to define:

- The concepts that exist in a domain where requirements are being specified. Provide basic terminology for requirements and stories.
- The particular attributes of these concepts.
- The relationships that exist between these concepts (generalization, aggregation, others more domain-specific).

Class diagrams are ideal to capture this information. However, a ***precise subset*** is useful here. Avoid unnecessary complex features of UML class diagrams!

Class Diagrams for Precise Domain Modeling

What to Include in Precise Class Diagrams:

1. **No operations (methods/functions)!**
   - The diagram should only describe the structure, not behavior.
2. **Use attributes with types**
   - For example: name: String, age: Integer
   - If the type is another class, use an **association** (a line connecting two classes).
3. **Associations must have precise multiplicities**
   - Like 1..* (one to many), 0..1, 1, etc.
   - This tells exactly how many instances are related.
4. **Name the association ends (roles)**
   - Instead of naming just the line between classes, label each end with the role it plays.
   - This helps with clarity when expressing rules (constraints).
5. **Use Enumerations**
   - When a value can only be from a fixed list (like days of the week or user roles), use an enumeration.
6. **Use constraints**
   - Add extra rules in words or using OCL (Object Constraint Language) to clarify conditions.
   - For example: "A person can only own up to 3 pets."

Class diagrams show structure (classes, attributes, relationships, multiplicity), but not **business rules** or **validity checks**— so we need **constraints**.

OCL (Object Constraint Language) Structure:

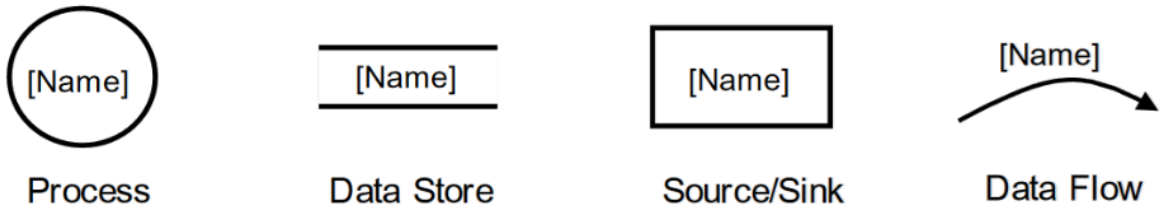
Each rule starts with:

- *context* **ClassName**: defines which class the rule applies to.
- *inv* **ConstraintName:** : declares a named **invariant** (a rule that must always be true)

Data Flow Diagrams (DFD)

- Data Flow Diagram or DFD is a graphical representation of the **flow of data through a system**.
- DFD illustrates How the data get processed in the systems in terms of **input and output**.
- It focuses upon the flow of the information, where data comes from, where data goes, and where it gets stored.
- DFD doesn't have control flow, and no loops or decision rules exist.
- DFDs are very popular because they help the developers and testers to visualize the major steps and data involved in software- system processes.

Components of DFD



- ***Processes*** in a system are functions that transform input data into output data.

- ***Data stores*** are abstract representations of persistent data that processes can both read from for input and write to for storing output.

- ***Sources/sinks*** are external, unchangeable entities that interact with a system by providing input data (*sources*) or receiving output data (*sinks*).

- ***Data flow*** represents the movement of data (both material and immaterial) in a system.

DFD Creation Rules

- Data cannot flow between two external entities.
- Data cannot flow between two data stores.
- Data cannot flow from an external entity to a data store.
- A process must have at least one *input* and one *output* data flow.
- A data store must have at least one *input* and one *output* data flow.
- Two data flows cannot cross each other.
- All the processes in the system should be linked to a minimum of one *data store* or any other *process.*
- A single DFD can have a maximum processes up to 9 and a minimum of 3 processes (for readability).

In short, **All data must pass through a process** before it goes **to or from**:

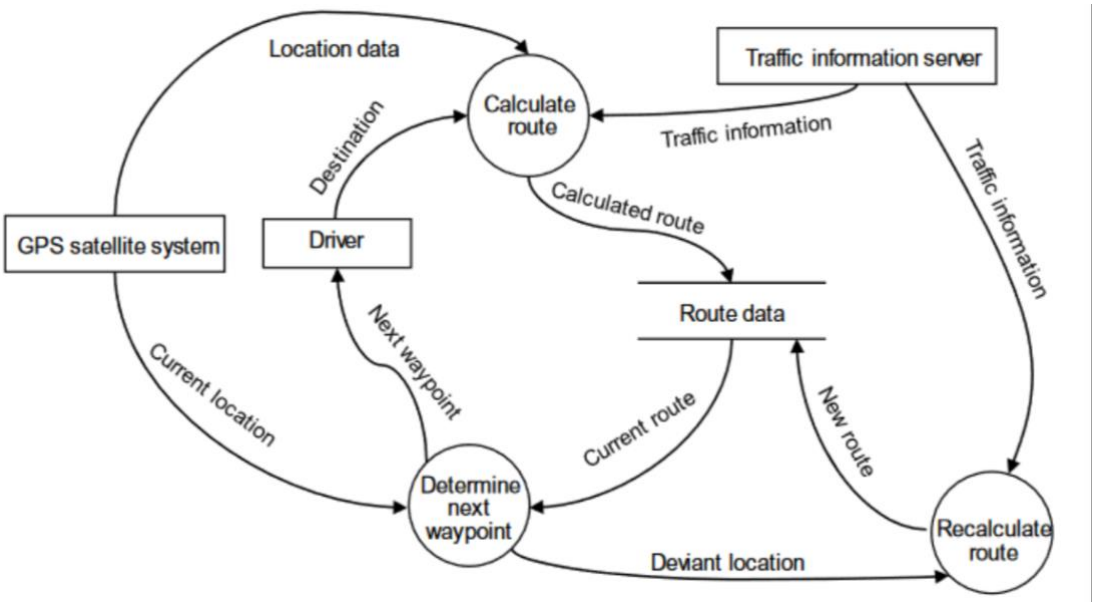
- an **external entity**
- a **data store**
- or **another data source/sink**

- **Input from the outside?** → Must be processed first.

- **Output to the outside?** → Must come from a process.

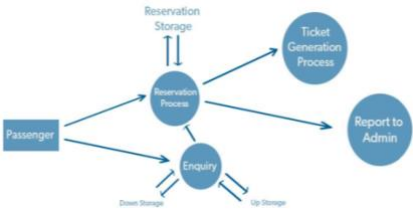
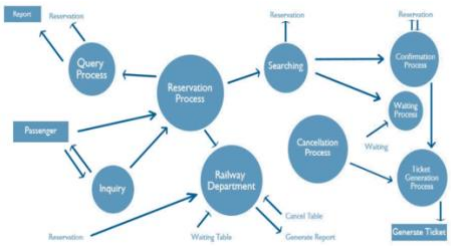
- **Data moving between stores?** → Needs a process in between.

DFD Example



DFD Levels

- Data Flow Diagrams can represent systems at different levels of detail, from a high-level overview to very detailed processes.
- Complex systems are decomposed into hierarchical levels in DFDs. Higher-level diagrams are partitioned into lower levels to reveal increasing detail and functional components.
- Levels in DFD are numbered 0, 1, 2 or beyond.
- Leveling helps manage complexity and makes DFDs easier to understand, analyze, and communicate.

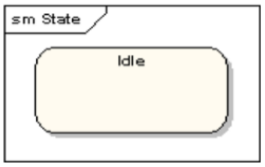
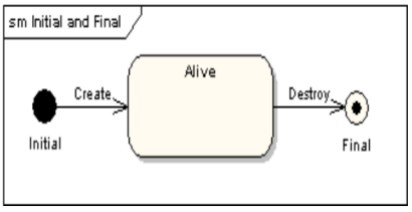
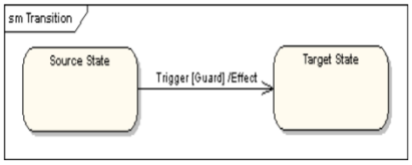
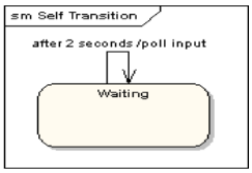
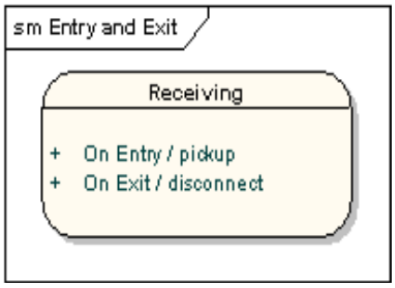
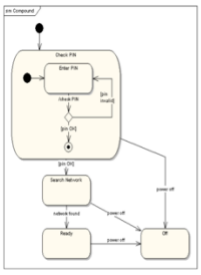
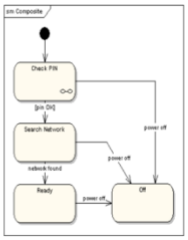
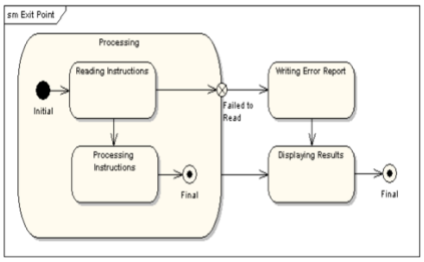
Level 0	Level 1	Level 2
Level 0 also known as ***Context Diagram***, designed to be an abstraction view Shows the system as a single process and its interactions with external entities.	Level 1 also known as ***Overview Diagram.*** Highlight the system's main functions and decomposes the high-level process from Level 0 into major sub-processes.	Level 2, ***Detailed Diagram,*** goes one step deeper breaking down Level 1 processes into more granular detail. It can be used to plan or record the specific/necessary detail about the system's functioning.
		

UML State Machine Diagram

- Depicts **data** and **behavior** of a **single object** throughout its lifetime.
- Set of states (including an initial start state).
- Transitions between states.
- Entire diagram is drawn **from that object's perspective.**
- What objects are best used with state diagrams?
  - Large, complex objects with a long lifespan.
  - Domain ("model") objects.
  - Not useful to do state diagrams for every class in the system!

Transitions:

- ***Trigger*** is the cause of the transition, which could be a signal, an event, a change in some condition, or the passage of time.
- ***Guard*** is a condition which must be true in order for the trigger to cause the transition.
- ***Effect*** is an action which will be invoked directly on the object that owns the state machine as a result of the transition.

State	A state is denoted by a round-cornered rectangle with the name of the state written inside it.	
Initial and Final States	The initial state is denoted by a filled black circle and may be labeled with a name. The final state is denoted by a circle with a dot inside and may also be labeled with a name. The initial state is denoted by a filled black circle and may be labeled with a name. The final state is denoted by a circle with a dot inside and may also be labeled with a name.	
Transitions	Transitions from one state to the next are denoted by lines with arrowheads. A transition may have a trigger, a guard and an effect.	
Self-Transitions	A state can have a transition that returns to itself. This is most useful when an effect is associated with the transition.	
StateActions	If the target state had many transitions arriving at it, and each transition had the same effect associated with it, it would be better to associate the effect with the target state rather than the transitions. This can be done by defining an entry action for the state. • It is also possible to define actions that occur on events, or actions that always occur. • It is possible to define any number of actions of each type.	
Compound States	A state machine diagram may include sub-machine diagrams. The ∞ symbol indicates that details of the Check PIN sub-machine are shown in a separate diagram.	 
Exit Points	• It is possible to have named alternative exit points. • Thefollowingdiagramgivesanexamplewherethestateexecuted after the main processing state depends on which route is used to transition out of the state	

(9) Requirements Prioritization and Agile Estimation

Prioritization Difficulties

- There are too many requirements!
- The requirements are from many different sources.
- Resources are limited (budget, time, expertise, etc.).
- Establishing priorities is important, but:

  - Which requirements are truly important, and to whom?
  - How to prioritize them? On what basis? What to minimize/maximize?
  - In which iteration should the requirement be considered?

- Developers may not know the business value of some requirements, and clients may not know the implementation complexity (or cost) of some requirements.
- Different stakeholders have different (and often conflicting) goals and different priorities.
- Some stakeholders' decisions carry more weight than others.
- Companies often lack systematic data, metrics, and technologies to support the prioritization process:
  - Often done manually, informally, on an ad-hoc basis
  - Difficult to establish and communicate

- Neglecting to prioritize requirements leads to rushed compromises and dropped features when deadlines approach, risking project quality and delivery.

Requirements Prioritization and Triage

- Requirements prioritization is also referred to as triage.
- Need to decide which requirements really matter, e.g., those that need to be implemented in the next release.
- Need for compromise, negotiation, priorities.
- Prioritization must help:
  - Make acceptable trade-offs among quality, cost, and time-to- market goals.
  - Allocate resources based on the importance of requirements to the project as a whole (project planning)
  - Determine when a requirement should become part of the product
  - Offer the right product!

The 80-20 Rule (a.k.a. Pareto Principle)

Pareto Principle in Business:

- 80% of a company's profits come from 20% of its customers.

- 80% of complaints often originate from 20% of customers.
- 80% of work is completed in 20% of the total available time.

- **20% of functionalities provide 80% of revenues.**
- The remaining 80% of functionalities offer a lower return on investment while adding delays, development costs, maintenance costs...
- How to find the most useful and beneficial 20% of functionalities and qualities?

Which Sector Should We Focus On?

Top-Left Shaded Region: "To Focus On!"

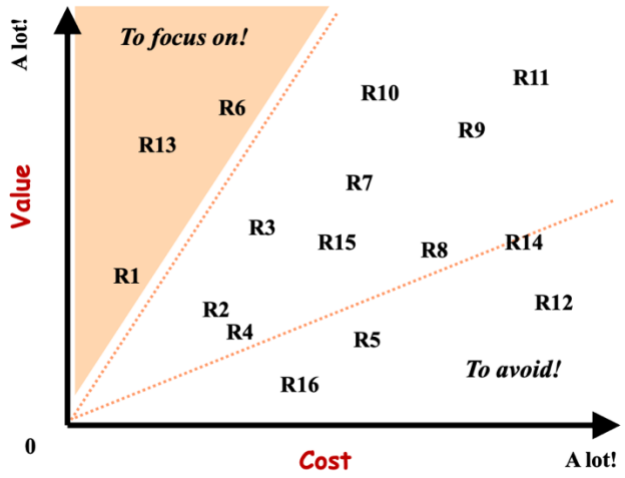
- Requirements like R6, R13, and R1 fall into this zone.
- These deliver high value at a low cost. Best investments — should be prioritized early.

Middle/Right Region: Mixed

- Includes R3, R7, R15, R10, etc.
- These offer moderate to high value but come with higher costs. May be considered after core priorities are addressed.

Bottom-Right: "To Avoid!"

- Requirements like R12, R14, and R16 lie here.
- They have high cost and low value. Not worth the investment — should be avoided.



Requirements Triage Process

Key Characteristics:

- Simple and fast: The process must be efficient enough for practical use in real-world industry settings.

- Accurate and reliable: Despite being quick, it should still produce trustworthy and consistent prioritization results.

What it should consider:

- Value to stakeholders (Maximize): Choose requirements that offer maximum benefit or impact to users and clients.

- Implementation cost (Minimize): Prefer requirements that are cheaper to build when possible.

- Time to market (Minimize): Prioritize features that can be delivered quickly, especially if speed gives a competitive advantage.

Important to agree on requirements granularity, E.g., user stories, features, detailed requirements...

Brainstorming: Idea Prioritization

No development team can implement everything that everyone thinks of.
Therefore, retained ideas must be prioritized.

Common Prioritization Techniques:

1. Cumulative Voting: The $100 Test.

2. "Critical, Important, Useful" Categorization.

Cumulative Voting: The $100 Test

This process is straightforward and usually works quite well.

- Each person is given **$100** of "**virtual idea money**" to be spent on "**purchasing ideas**".

- Each participant decides how much money to spend on each idea. Then votes are tabulated, and the results are rank ordered.

- A quick histogram of the result can help participants see the visual impact of their decisions.

Caveats:

- It may be necessary to **limit the amount** anyone spends on one feature. Otherwise, a participant, might put all of their money on a single feature in order to elevate it to a higher priority.

- It may work only once, because once the results are known, participants may bias their input for the second vote.

"Critical, Important, Useful" Categorization

Everyone votes on every idea — but must classify each one fairly across three levels of importance. (Each stakeholder is given only one- third of the votes from each category).

- **Critical** features are mandatory: a system deployed without this feature could not fulfill its primary mission or meet the market need.

- **Important** means that failure to include the feature could cause a loss of customer utility, market share, or revenue.

- **Useful** means nice to have. Useful features make the system more appealing to use or deliver higher utility to some class of users.

- After voting, each feature will probably have a mix of categories.

- Multiply "critical" votes by 9, "important" by 3, and "useful" by 1; then add up the score.

- This tends to spread the results to heavily favor the "critical" votes, and thus every stakeholder's "critical" need will tend to bubble toward the top.

Features

Features can be described as follows: **"Services provided by the system that fulfill one or more stakeholder needs."**

Marketing language: Features are often expressed in terms that highlight benefits and value to customers — not just technical functions.

Agile organization: In agile development, teams are often structured around features, known as feature teams, to focus on delivering complete, user-valued functionality.

Business impact: A challenge in prioritization is figuring out how much a single feature contributes to maintaining or increasing market share, revenue, or customer satisfaction.

Prioritizing Features

In practice, teams often have to compare very different types of features, such as:

A large, complex feature that takes months to develop vs. a small, quick feature that can be done in a few weeks.

How is this done?

Return on Investment (ROI)

- By estimating the Return on Investment (ROI) for each feature: Predict how much revenue, user value, or market gain the feature is likely to generate once released.

- Product managers and business analysts usually only propose features they believe will bring high value. They wouldn't spend time on a feature unless they could justify it with a strong Return on Investment (ROI). In other words, if they didn't believe the feature would benefit the business, they wouldn't have pursued it at all.

The relative ROI is the relationship between potential return (value) divided by the effort (cost to implement) for a feature.

Deliver a higher ROI feature before a lower ROI feature.

What's Wrong with Value/Effort ROI?

- Based on a more complete economic framework, the assumption that a high relative ROI feature should naturally have precedence over a lower ROI feature is not Correct.

- The potential profit for a particular high ROI feature could be less sensitive to a schedule delay than a lower ROI feature.

Cost of Delay (CoD)

- **Cost of Delay (CoD)** refers to the **loss in value** that occurs when a feature is **delayed**.
  Reinertsen outlines three economic-based methods to prioritize work using CoD:

1. Shortest Job First (SJF)

When multiple features have equal delay costs, prioritize the one that takes the least time to complete.
Why? It reduces overall delay cost faster and improves flow efficiency.

2. High Delay Cost First
If features require similar effort, do the one with the highest cost of delay first.
Focuses on value urgency — prevents high-value losses due to delay.

3. Weighted Shortest Job First (WSJF)
A combined approach:
WSJF = Cost of Delay / Job Duration (effort)
Prioritize features with the highest WSJF score.
Balances both urgency (CoD) and implementation time.

Estimating the Cost of Delay

CoD is an aggregation of three attributes of a feature:

User value: is simply the potential value of the feature in the eyes of the user (relative estimate).

Time value: is another relative estimate, one based on how the user value decays over time.

Risk reduction/opportunity enablement value: acknowledges that some features are more or less valuable to us based on how they help us mitigate risk and help us exploit new opportunities.

Prioritization is Local, Global, and Temporal

This concept emphasizes that prioritization decisions depend on multiple perspectives and change over time:

1. Cost of Delay (CoD) – Global
Reflects the overall business impact of delaying a feature. It applies across the organization, regardless of who builds the feature.

2. Effort – Local
Refers to how much work is needed by the specific team developing the feature. Different teams may take different amounts of time for the same task.

3. Temporal – Time-Sensitive
Priorities shift over time as deadlines approach. A feature may not seem urgent now but becomes critical later.

Example: A new student registration system:

January: Low urgency (CoD = 1–2)
May: Very high urgency (CoD = 10) as the deadline nears

Priorities should be set "at the last responsible moment" — the latest point when decisions can be made wisely.

Achieving Differential Value: The Kano Model of Customer Satisfaction

The Kano Model is a framework used to categorize requirements based on how they affect customer satisfaction. It helps teams prioritize features that will maximize user happiness and avoid negative reactions. The Five Kano Categories:

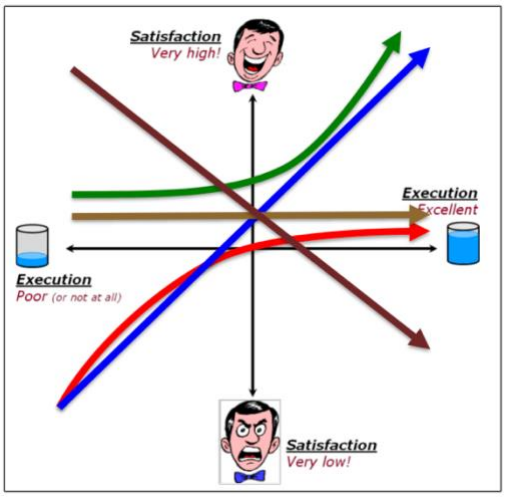
1. **Basic Requirements (Red)**
Expected and taken for granted by customers.
If missing, customers are dissatisfied.
If present, satisfaction does not increase much.

2. **Performance Requirements (Blue)**
Directly requested by customers.
The better they are implemented, the more satisfied the customer.

3. **Excitement Requirements (Green)**
Unexpected features that delight customers.
Not delivering them doesn't hurt, but including them greatly boosts satisfaction.

4. **Indifferent Requirements (Brown)**
Customers don't care whether they exist or not.
These add no real value to satisfaction.

5. **Reverse Requirements (Orange)**
Features that annoy or frustrate customers.
The more you implement them, the worse the experience.



Estimating User Stories Using Story Points

Story points are used in agile development to estimate the size of a user story — not in hours or days, but in terms of relative effort.

What Do Story Points Represent? A story point is an integer value that reflects a combination of:

Complexity – How difficult is it to implement?
Volume – How much work is involved? How long might it take?
Uncertainty – How unclear or risky is the requirement?

Key Characteristics:

- Unit-less: Story points don't represent specific time (e.g., hours or days).
- Relative scale: A 2-point story is expected to take twice as much effort as a 1-point story, and so on.

Why It's Useful:
This approach helps teams estimate based on experience and intuition, rather than fixed time, allowing for flexibility and better planning in agile environments.

Planning Poker

Planning Poker is a team-based agile estimation technique that promotes discussion, consensus, and accuracy in estimating user stories.

Key Principles:

- Begin with preliminary design discussion to clarify understanding.
- Keep the session within a short timebox (around 30 minutes).

Subtle Aspects of the Technique:

Team-Based Estimation:

- All team members (including developers and testers) estimate together.
- Ensures both implementation and testing perspectives are considered.

Modified Fibonacci Scale (0, 1, 2, 3, 5, 8, 13, 20, 40, 100):

- Smaller numbers are close together to allow precision for small stories.
- Larger numbers have bigger gaps to reflect growing uncertainty.

Handling Large Stories:

- If a story gets an estimate of 20, 40, or 100, it's likely too large for a single iteration.
- These should be split into smaller stories.

Estimation Dynamics:

- 0 Points are used for stories that are trivial or already done.
- After discussion, everyone shows their card at the same time: Prevents bias or anchoring based on others' opinions.
- If estimates vary, the high and low estimators explain their reasoning.
- Teams then re-estimate to reach consensus.

Efficiency Tips:

- Spend 2–5 minutes per item.
- Teams can estimate 10–20 stories per hour.
- This makes it a fast and effective planning method.

Planning Poker combines team collaboration, structured estimation, and discussion-driven refinement to produce reliable story point estimates while enhancing shared understanding.

Tabletop Relative Estimation

Tabletop Relative Estimation is a visual and collaborative technique for estimating user stories based on their relative size, rather than assigning exact point values upfront.

How It Works:
Conducted with face-to-face communication. The team discusses each story and places it on a table based on its relative size:
Small stories go to the left.
Larger stories go to the right.
Stories of similar size are stacked in columns.

Stories may be rearranged (shuffled) as discussions evolve.

Key Benefits:
Allows team members to compare stories side by side.
Emphasizes relative sizing, not exact numbers.
Estimates come later — point values (e.g., 1, 3, 5, 8…) are assigned to columns, not individual stories.
Provides a clear visual overview of the workload for the upcoming iteration.

Why It's Useful:
This method promotes shared understanding, supports faster alignment, and gives the team a big-picture view of story sizes and scope — making planning more intuitive and collaborative.

From Scope Estimates to Team Velocity: Establishing Velocity

Team's velocity: How many points the team can complete in a standard iteration (sprint).

From Velocity to Schedule and Cost: Estimating Schedule

If we know the size and velocity, we can calculate how long it will take to complete a story.

Estimating cost: simply
1. take the average burdened cost for a team and divide it by their velocity.
That provides the cost per story point for that team.

2. Then when the team estimates an arbitrary backlog, just multiply the cost per story point for that team by the total estimate for the backlog.

Caveats on the Relative Estimating Model

- It is based on historical data and is predictive only to the extent that the future (new stories) looks like the past (stories already completed).

- It is valid only to the extent that the team continues to have the same individuals. If you change the team members (for example, if we doubled the size of the team), velocity will change dramatically, but it should stabilize after few iterations.

- A team's velocity cannot be compared to any other team. Imagine if team 1 had used 2 as the smallest story and compared everything to that. Their apparent velocity would be twice as large, but the actual productivity would be the same.

Increasing Velocity, Be Careful What You Ask For

- The goal is to continuously increase velocity while improving quality at the same time.

- The ability to achieve a certain amount of functionality in a time period, is not a complete measure of productivity.

- Velocity is only a tool by which teams manage and measure themselves. If management attempts to use velocity as a measure of team performance, the team will respond in one of three ways:

    1. Continuously improve the team's true productivity and agility in all aspects.

    2. Cut back on quality.

    3. Simply increase the size of the estimates.

Problems with Estimating using story points

1. Not easy to understand by the team, and it's even less easy to understand by their outside stakeholders.

2. It's hard to get started. Until teams have done a few iterations, they have no idea how to predict what they can accomplish.

3. Getting to schedule and cost estimates is very indirect: work through relative estimates, establish velocity, and so on, and you have to understand the burden cost of each individual team before you can translate a story point into a cost.

4. Teams occasionally struggle to adjust their velocity based on the availability of team members. For example, if a team member is only part-time for a sprint or a key resource is unavailable for a period, what is the anticipated velocity then?

5. Team velocities are not normalized. It's not unusual for one small team to have a velocity of 40 points per iteration, while a team twice that size has a velocity of half that. That makes for some pretty uncomfortable discussions.

(10) Requirements Conflicts and Negotiation

Conflicts in RE

A conflict in **requirements engineering** exists, if the **needs and wishes** of different stakeholders (or groups of stakeholders) regarding the system **contradict each other**, or if some **needs** and wishes **cannot be taken into account**.

Possible Conflicts among Stakeholders

Conflicts related to interests, structure, goals, features, requirements, beliefs, cultures,...

Examples:
- Conflicts between suppliers and customers about costs, benefits, risks...
- Power struggle within customer organization.
- Conflicts with other projects about resources (availability but also control).

In practice, causes are frequently intermixed.

Impact of Conflicts (in Requirements Engineering)

- Unresolved conflicts **compromise the acceptance** of the system by the stakeholders.
  - As a result, some stakeholders may **no longer support the development** of the system, or
  - the conflicts may even cause **a failure of the development project**.
- Conflicts should not be regarded **negatively** in requirements engineering.
  - They can serve as **a source of new ideas and the development of innovative requirements.**

Requirements Negotiation Activity

Negotiation in requirements engineering aims to achieve agreement among all stakeholders about the requirements. Identifying, analyzing, and resolving the conflicts.

Requirements negotiation is **one core activity of requirements engineering.**

Conflict Management Resolution

1. *Identify* conflicts.
2. *Analyze* the cause of each conflict.
3. *Resolve* the conflicts by means of appropriate strategies.
4. *Document* the conflict resolution and its rationale.

(1) Identifying Conflicts

- Conflicts do not always become obvious: **A stakeholder may not express his disapproval of a requirement immediately**.
- Requirements engineers should **pay special attention** during all requirements engineering **activities to detect and document (possibly latent) conflicts.**
- Requirements engineers need to **actively search for conflicts** that have remained undetected during the other requirements engineering activities.

(2) Analyzing Conflicts

Determine the type of conflict:

- ***Data conflict:*** A data conflict is caused by a lack of information, by misinformation, or by different interpretations of an issue.
- ***Interest conflict:*** An interest conflict is caused by subjectively or objectively different interests or goals of stakeholders.
- ***Value Conflict:*** A value conflict is caused by different criteria that stakeholders apply when evaluating an issue (e.g., cultural differences).
- ***Relationship conflict:*** A relationship conflict is caused by negative interpersonal behavior among stakeholders (e.g., disrespect, insulting).
- ***Structural conflict:*** A structural conflict is caused by an unequal balance of power between stakeholders.

Heuristics for Conflict Analysis

1. Checking for a data conflict:

- Let the stakeholders explain the conflicting requirements.
- Does one of the explanations **deviate from the actual requirement?** If so, a data conflict exists.

2. Asking for the stakeholders' interests:

- Ask the stakeholders for their **goals** with regard to the conflicting requirements.
- Check the resulting **goals for contradictions**. If there is a contradiction, an interest conflict exists.

3. Clarifying the stakeholders' evaluation backgrounds:

- Ask the stakeholder why he/she evaluates the conflicting requirements in the way he/she does.
- Check **the evaluation backgrounds** of the different stakeholders for differences that may cause the detected conflict. If there is such a difference, a value conflict exists.

(3) Resolving Conflicts

To resolve a conflict, one of the following **three basic strategies** can be applied:

Strategy	Definition	Advantage	Diadvantage
Negotiation	The conflicting parties agree on a solution by means of negotiation.	the viewpoints of all conflicting parties are considered, and a win-win situation is created.	1. Can be a very time consuming process. 2. The compromise may not be the best solution from an objective viewpoint.
Creative solution	The original viewpoints of the conflicting parties are discarded, and a new creative solution is developed that harmonizes the viewpoints of all conflicting parties.	All conflicting parties come off as winners, since a solution is found that is acceptable to all parties.	1. Can be time consuming. 2. May have an impact on other requirements that are influenced by the solution.

Decision	A higher authority makes a decision in favor of one conflicting party.	Quick resolution without consuming too many resources	1. A conflict can only be resolved by a higher authority (if exists). 2. When a decision is made in favor of one viewpoint, the other viewpoint is ignored. This can negatively influence the motivation of the ignored, conflicting party.

Resolving Data Conflicts

- A data conflict is based on the misinformation of stakeholders or on misinterpretation.
- Should be resolved through negotiation. During negotiation, information exchange between the involved, conflicting parties takes place.
- The conflicting parties may be provided with additional, relevant information.
- The use of creative solution is not suited for data conflicts because it would be based on incomplete or incorrect information.
- Decision-making in favor of one conflicting party is not suited. Decision could be made in favor of the wrong viewpoint due to missing information or a misunderstanding.

Resolving Interest Conflicts

- Based on the common interests, the parties should first try to resolve the conflict through negotiation.
- If an agreement is reached, each conflicting party should benefit from the resolution of the conflict.
- A decision-maker is recommended for: Conflicts that cannot be resolved by means of negotiation. OR the conflicting parties are not willing to find a compromise at all.
- The creative solution strategy is only conditionally suited, because the main problem is the difference of goals or interests, which have to be harmonized or substituted by new goals/interests before a creative solution can be found that is accepted to all parties.

Resolving Value Conflicts

- **Negotiation** is only **conditionally suitable** as a strategy for resolving value conflicts, since **values are often deeply rooted in the stakeholders' personality**. These values **cannot be easily changed or abandoned during a negotiation** about the conflict.
- Requirements engineers should **strive for a creative solution**. One example of a creative solution for a value conflict is to realize different variants of the system so that each stakeholder finds his or her values respected by some system variant.
- However, it should only be considered if the value conflict is of **significant importance** for the system and **there are sufficient resources** for implementing the solution.
- If the values of the conflicting parties are irreconcilable, **the conflict should be resolved through a decision.**

Evaluation of the Conflicts Resolution Strategies

	Negotiation	Creative Solution	Decision
Data Conflict	Suitable	Not Suitable	Not Suitable
Interest Conflict	Suitable	Conditionally Suitable	Suitable
Value Conflict	Conditionally Suitable	Suitable	Conditionally Suitable

(4) Documenting Conflicts Resolution

- Documentation of **achieved conflict resolutions** together with the **viewpoints** of the conflicting parties and the **pros** and **cons** of each viewpoint is recommended.
- If conflict resolutions are **not documented**, the achieved solutions **may be forgotten** and conflicts that have already been resolved **may recur**.
- For instance, some stakeholder might ask for a requirement that is in conflict with another requirement and has therefore been **discarded earlier during conflict resolution**.

Negotiation Techniques: Win-Win Approach

The goal of the Win-Win [Boehm and Ross 1989] approach is to make all stakeholders become winners.

The Win-Win approach differentiates three basic situations:

- ***Win-Lose:*** If some stakeholders achieve their goals in a conflict **at the expense of other stakeholders**. Some stakeholders are the **winners**, other stakeholders are the **losers** of the conflict.

- ***Lose-Lose:*** If **no conflicting party achieves its goals** in a conflict, this situation is referred to as a lose-lose situation. In this case, all involved parties are losers

- ***Win-Win:*** If all conflicting parties **achieve their goals completely or partially in a conflict, all parties are winners**.

- During conflict resolution, the stakeholders should always strive for a win-win situation.

- **Understand how stakeholders want to win***:* the requirements engineers must know **what each stakeholder considers as a benefit for himself**.

- Resolution of a conflict by creating a win-win situation provides **confidence among the stakeholders** and also **increases their willingness to make a compromise.**

- If a conflict is resolved through **negotiation** or through a **creative solution**, all stakeholders should benefit from the resolution of the conflict.

- Resolving a conflict through a **decision** generally leads to a **win-lose** situation since the decision is typically made in favor of a single viewpoint and hence privileges one party.

(11) Requirements Validation

Requirements Risks

Requirements can be inadequate in many ways including:

- Inaccurate or incomplete stakeholder identification.
- Incomplete, inconsistent, or incorrect requirements.
- Incorrectly ranked requirements.

Requirements risk management involves the ***proactive analysis***, ***identification***, ***monitoring***, and ***mitigation*** of any factors that can threaten the integrity of the requirements engineering process.

Requirements Verification and Validation

- Requirements validation and verification involves ***review***, ***analysis***, and ***testing*** to ensure that a system complies with its requirements.
- Compliance pertains to both ***functional*** and ***nonfunctional*** requirements.

Validation vs. Verification

Validation	***Verification***
"Are we building *the right* product?"	"Are we building the product ***right***?"
1. Check that the right product is being built. 2. Ensures that the software being developed (or changed) will satisfy its stakeholders' real needs. 3. Checks the software requirements specification against stakeholders' goals and requirements.	1. Check that product is being built right. 2. Ensures that each step followed in the process of building the software is well done. 3. Checks consistency of the software requirements specification artefacts and other software development products (design, implementation, ...) against the specification.
- Both need to be performed at every stage during the (requirements/development) process.	

Requirements V&V vs. Requirements Analysis

- Both have several activities in common.
  - Reading requirements, problem analysis, meetings, discussions...
- But inputs are different!
- Analysis works with raw, incomplete requirements as elicited from the system stakeholders.
  - Develop a software requirements specification document.
  - Many decisions by requirements analysts need confirmation.
- Requirements V&V works with a software requirements specification containing a presumably complete and correct set of requirements.
  - Check that this specification is accurate.
  - Emphasis on "we have the *right* requirements *well done*".

Requirements V&V Techniques

1. Simple checks.
2. Prototyping.
3. Functional test design.
4. Formal V&V.
5. Reviews and inspections.
6. Development of user manual.

1. Simple checks

- Various checks can be done using traceability techniques.
  - Given the SRS, verify that all elicitation notes are covered adequately.
  - Often involves developing traceability matrices.
  - Tracing is also done between different levels of requirements. Checking requirements and features against goals and user stories...
  - Completeness and consistency checks.
- Verify that the requirements are well-written
  - According to the criteria already discussed in the standards and the course.
  - With or without tool support.

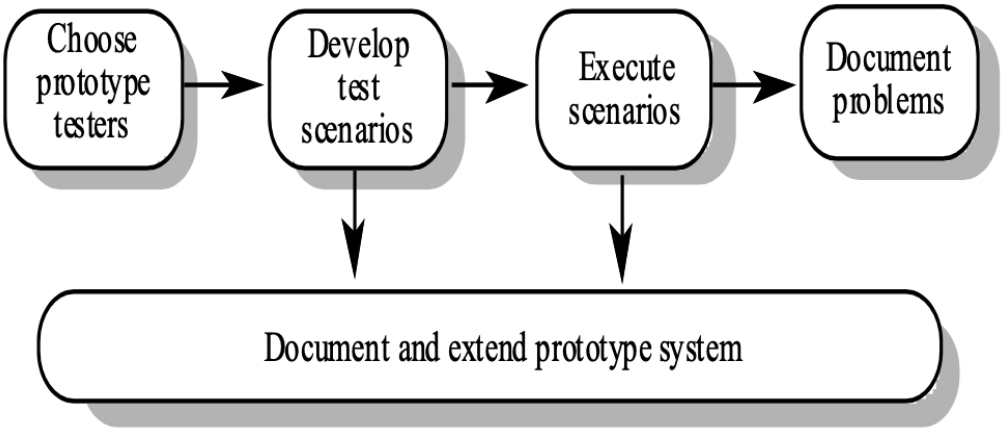
2. Prototyping

Dilema: You can't evaluate design until it's built But...
- After building, changes to the design are difficult.
- Simulate the design, in low-cost manner.

Excellent for validation by users and customers

- Prototypes for requirements validation demonstrate the requirements and help stakeholders discover problems.
- Validation prototypes should be complete, reasonably efficient and robust.
- It should be possible to use them in the same way as the required system.
- User documentation and training should be provided.

Prototyping for Validation



Prototyping Validation Steps

1) ***Choose prototype testers***
The best testers are **users** who are fairly **experienced** and who are **open-minded** about the use of new systems.

2) ***Develop test scenarios***
Careful planning is required to draw up a set of test scenarios which provide **broad coverage** of the requirements. End-users shouldn't just play around with the system as this may never exercise critical system features.

3) ***Execute scenarios***
The users try the system by executing the planned scenarios.

4) ***Document problems***
Its usually best to define some kind of electronic or paper problem report form which users fill in when they encounter a problem.

3. Functional test design.

It involves creating tests based on the ***functional requirements*** of a system — i.e., what the system should do.

- Functional tests must eventually be created at the system level, whether early or late in the project.

- These tests can be derived directly from the requirements specification, meaning:
  - If a requirement says "The system shall allow login," a functional test would verify that.

- Designing tests early can help uncover errors or ambiguities in the requirements — even before coding starts!
  - Example: If a requirement says "respond quickly," what does "quickly" mean exactly? This vagueness makes test design hard.

- Missing or unclear requirements make it difficult to define precise, testable conditions.

- In ***agile development***, testing may begin before programming starts. This is called **Test-Driven Development (TDD)**:
  - You **write a test first** (based on what the code should do),
  - Then **write the code** to pass that test,
  - Finally, **refactor** the code to improve quality.
  - (The diagram on the right illustrates this cycle: Test ➜ Code ➜ Refactor)

4. Formal V&V

Formal V&V involves using ***mathematical models and tools*** to verify and validate system behavior. The goal is to ensure a system ***does what it's supposed to do***, according to its formal specification.

For example, the following may be provided for ***Finite State Machines (FSMs)*** through tools:

- ***Simulation***: random or interactive.
- ***Testing***: predetermined traces have to be accepted (or rejected).
- ***Model checking***: given a model M and some properties P, show that any system implementation satisfying M will have the properties P

5. Reviews and Inspections

- A group of people read and analyze requirements, look for potential problems, meet to discuss the problems (physically or online), and agree on a list of action items needed to address these problems.
- Widely used V&V techniques Lots of evidence of effectiveness of these techniques
- Can be expensive if we are not careful
  o Planning and preparation
  o Individual pre-review checking
  o Need appropriate checklists
  o Must be developed if necessary, and maintained

Reviews and Inspections Types

- Different types of reviews with varying degrees of formality
- Walkthroughs: Can be led by author/expert to educate others on his/her work
- Reading and approval (sign-off): Encourages the reader to be more careful (and responsible!)
- Formal inspections
  Structured and detailed review, defined roles, preparation, etc.
  Example: Fagan Inspection

Fagan Inspection

- Characterized by rules on who should participate, how many reviewers should participate, and what roles they should play
  - Not more than 2 hours at a time, to keep participants focused o 3 to 5 reviewers.

- Metrics are collected.
- ***Author*** serves as the presenter of the document.
- ***Important:*** **the *author's supervisor*** does not participate in the inspection and does not have access to data. This is not an employee evaluation.
- ***Moderator*** is responsible for initiating the inspection, leading the meeting, and ensuring issues found are fixed.
- All reviewers need to prepare themselves using checklists o Issues are recorded in special forms.

- The inspection meeting is like a brainstorming session to identify (potential) problems
- Re-inspection if > 5% of the document change
  - Some variants are less tolerant... too easy to introduce new errors when correcting the previous ones!

Review – Typical Problem Categorization

- **Requirement clarification**
  The requirement may be badly expressed or may have accidentally omitted information which has been collected during requirements elicitation.
- **Missing information**
  Some information is missing from the requirements document.
- **Requirements conflict**
  There is a significant conflict between requirements.
  The stakeholders involved must negotiate to resolve the conflict.
- **Unrealistic requirement**
  The requirement does not appear to be implementable with the technology available or given other constraints on the system.
  Stakeholders must be consulted to decide how to make the requirement realistic.

Requirements Review Checklists

- **Essential tool for an effective review process**
  List common problem areas and guide reviewers
  May include questions on several quality aspects of the document: *comprehensibility,*

redundancy, completeness, ambiguity, consistency, organization, standards compliance, traceability ...

- **There are general checklists and checklists for particular modeling and specification languages.**

- **Checklists are supposed to be developed and maintained**

They evolve over time: some points that do not detect issues can be removed while new points may be added to check for emerging categories of issues

Comments on Reviews and Inspections

Benefits

- Effective (even after considering cost).
- Allow finding sources of errors (not only symptoms).
- Requirements authors are more attentive when they know their work will be closely reviewed.
- Encourage them to conform to standards.
- Familiarize large groups with the requirements (buy-in).
- Diffusion of knowledge.

Risks

- Reviews can be dull and draining (need to be limited in time).
- Time consuming and expensive (but usually cheaper than the alternative).
- Personality problems.
- Office politics...

6. User Manual Development

Writing a user manual from the requirements **forces a detailed requirements analysis** and thus can reveal problems with the document

Information in the user manual:

- Description of the functionalities
- How to get out of trouble
- How to install and get started with the system

(12) Fundamentals of Requirements Management

Sources of change

Requirements Change is Inevitable.

- Better understanding of the domain: new requirements become apparent.

- Everything else is changing (business, context, technologies, markets...).
  - Changing customer priorities, new needs, new competitors.
  - Collaborating systems may change.
  - Technical or business opportunities (cloud, IoT, blockchains...).

- Changing laws, regulations, polices, etc.

- Requirements errors, conflicts, and inconsistencies.

- Technical, schedule, or cost problems

- Change is as difficult to control as it is inevitable!

- Possible responses to change
  - Ignore it...
  - Add, modify, or remove requirements

Some Problems Due to Changing Requirements

- Requirements changing towards the end of development without any impact assessment Also known as *requirements/feature creep*.

- Unmatched/outdated requirements specifications causing confusion and avoidable rework "Which version is the right one already?"

- Time spent coding, writing test cases or documentation for requirements that no longer exist.

Requirements Management

A systematic approach to *eliciting*, *organizing*, and *documenting* the requirements of the system, and a process that establishes and maintains agreement between the customer and the project team on the *changing requirements* of the system.

Requirements Management Activities

Requirements management includes all activities intended to maintain the integrity and accuracy of expected requirements

- Manage changes to baseline (increments).
- Keep project plans synchronized with requirements.
- Control versions of individual requirements and versions of requirements documents.
- Manage relationships between requirements.
- Managing the dependencies between the requirements and other documents/artefacts produced in the systems engineering process.
- Track requirements status

Requirements Stability and Volatility

Volatility: Measures the number of requirements changes (addition, deletion, modification) in a specified time interval.

Stable requirements are concerned with the essence of a system and its application domain.

- Derived from the client's principal business activities or the domain model.
- Example: **a hospital will always have doctors, nurses, patients...**

Volatile requirements are specific to the instantiation of the system in a particular environment for a particular customer at a particular time. In other words, Requirements that are true for now, here, and for this user, but may change.

Expectations of Requirements Management

- ***Identification*** of individual requirements (**Unique Identification**).

- ***Traceability*** from highest level requirements to implementation
  - Established via links through a requirements database.
  - Links between requirements and design models, tests, code...
  - Coverage and consistency analysis

- ***Impact assessments*** of proposed changes
  - Which other requirements (and other linked artifacts) will be affected by a change.

- ***Controlled access*** to current project information

- A shared database ensures that all users are working with current data (consistency, parallel access, unique source of truth)
- A central repository allows all users to see the information that they need to see (visibility)

- ***Change control*** is a way to manage changes in a project in an organized and safe way. It makes sure that no one makes random changes without checking if it's okay first.

- Deployment of required ***tool support*** Use the right tools to help with requirement tracking, updates, and collaboration.

Identification of Requirements

- It is essential for requirements management that every requirement has a *unique* identification.

- Problem with a hierarchical approach (2.1, 3.2.4, ...):
  - Numbers (dynamically updated) cannot be assigned uniquely and definitively if the document is restructured

- Problem with symbolic identification: Requirements can be identified by giving them a symbolic name which is associated with the requirement itself (e.g., SEC1, SEC2, SEC3... may be used for security requirements). But... it is difficult to give only one category to a requirement

- Conclusion: *avoid encoding too much information in the ID*! ***A simple unique integer number suffices!***

Requirements have Attributes

- Attributes establish **context** and **background**, and go beyond the requirement description
- For filtering, analysis, metrics...
  - Creation date, Last update, Author, Stakeholders (Owners / Source)
  - Version number
  - Status, Priority, Importance, Stability
  - Rationale, Comments
  - Acceptance criteria
  - Subsystem / Product release number

- The more complex the project, the richer the attributes...
- Many attributes are predefined in RM tools, others are defined by requirements engineers as required by the project.

Requirements Change Status

Help manage the requirement lifecycle. Their number and nature depend on the process in place.

Examples of statuses:

- ***Proposed***: by some stakeholder
- ***Approved***: part of baseline, committed to implement
- ***Rejected***: after evaluation
- ***Implemented***: designed and implemented
- ***Verified***: Relevant tests have passed
- ***Deleted***:Removedfromlist

Change Request Form

Proposed changes are usually recorded on a change request form which is then passed to all people involved in the analysis of the change

Change request forms may include:

- Date, Customer, Requester, Product including version
- Description of change request including rationale
- Fields to document the change analysis
- Signature fields
- Status
- Comments

Version Control

Every version of a requirement needs to be uniquely identified

- Changes need to be documented and clearly communicated
- A version identifier must be updated with every change to the requirement

Requirements documents should include

- A revision history: changes, dates, by whom, why, etc.
- Standard markers for revisions (e.g., strikethrough or underlined text, coloring, line markers)

Version control tools may be used

- To store and manage the revision history
- To store justifications (to add, modify, delete, reject a requirement)

Traceability

- A link or relationship defined between entities

- Requirements traceability refers to the ability to describe and follow the life of a requirement, in both forwards and backwards direction (i.e., from its origins, through its development and specification, to its subsequent deployment and use, and through all periods of ongoing refinement and iteration in any of these phases)".

- One cannot manage what cannot be traced.

- Traceability is often mandated by contracts and standards, e.g., military and aerospace.

- Traceability gives essential assistance in understanding the relationships that exist within and across software requirements, design, and implementation.

Benefits of Traceability

- Prevents losing knowledge.
- Supports the verification process (certification, localization of defects)
- Impact analysis
- Change control
- Improved software quality (make changes correctly and completely)
- Reuse (by identifying what goes with a requirement: design, code...)
- Risk reduction (e.g., if a team member with key knowledge leaves.

Traceability Difficulties

- Various stakeholders require different information.
- Huge amount of requirements traceability information must be tracked and maintained.
- Integrating heterogeneous models/information from/to different sources (requirements, design, tests, code, documentation, rationales...) is not trivial.
- Manual creation of links is very demanding (likely the most annoying problem)Specialized tools must be used.
- Requires organizational commitment (with an understanding of the potential benefits)

Backward and Forward Traceability

Backward traceability

- To **previous** stages of development.
- Links other documents (which may have preceded the requirements document) to relevant requirements.

Forward traceability

- Links requirements to the design and implementation components.
- Help assure that all requirements have been satisfied.

Representation – Traceability Matrix

- Define links between pairs of elements, e.g., requirements to requirement, use case to requirement, requirement to test case...
- Can be used to define relationships between pairs, e.g., specifies/is specified by, depends on, is parent of, ...
- More amenable to automation than traceability table.

Representation – Traceability List

Traceability matrices become more of a problem when there are hundreds or thousands of requirements as the matrices become large and are sparsely populated.

A simplified form of a traceability matrix may be used where, along with each requirement description, one or more lists of the identifiers of related requirements are maintained.

Traceability Planning

- Planning traceability? Yes, just like any other project!
- Who are the stakeholders?
- What are the needs (analysis, reports...)? Useful, measurable, feasible objectives
- Definition of links and attributes. Can some be inferred automatically?
- Process (who collects and when to collect traceability information) Roles, access, Data/link input and updates Exceptional situations (e.g., lack of time).
- Traceability policies define what and how traceability information should Representations, queries, tools be maintained.

Factors to Consider during Planning

- ***Number of requirements***, the greater the number of requirements, the more the need for formal traceability policies.

- ***Expected system lifetime*** (e.g., F-18 jet fighter), more comprehensive traceability policies should be defined for systems which have a long lifetime

- ***Maturity level of organization.***

- ***Size of project and team.***

- ***Type of system*** (e.g., critical vs Web).

- ***Additional constraints*** from customer, e.g., compliance to military standard.

Modeling Traceability

- The types of links to use (and their attributes) must be defined for different types of requirements. It is a design problem!

- May be modeled with a UML class diagram (metamodel)
  - Object types (classes).
  - Object attributes (typed attributes).
  - Structure of folders, modules, objects. Stereotypes, composition...
  - Link types (associations): Satisfies, tests, refines, contains, contributes to, threatens, justifies...
  - Link attributes (association classes) Link cardinalities/multiplicities.

Tools to document traceability Information

- General purpose tools (e.g., spreadsheet programs, word processors, hypertext editors).
- Suitable for small and short term projects.
- Not sufficient for extensive requirements tracing purposes.

Requirements management tools

- Suitable for projects producing large and complex systems.
- Require investments (e.g., licenses, training end-users, system maintenance, consultation)
- Examples: Rational RequisitePro, DOORS

What are Suspect Links?

- Proactively know when changes impact your requirements.

- Suspect link indicates that an element may have been affected by a change.

- Help ensure that ripple effects of changes are considered.

(13) Prototyping

Definitions

"Software prototyping is the activity of creating prototypes of software applications, i.e., **incomplete versions of the software program being developed**. It is an activity that can occur in software development and is comparable to prototyping as known from other fields, such as mechanical engineering or manufacturing. **A prototype typically simulates only a few aspects of, and may be completely different from, the final product.**" *Wikipedia*

"The IEEE defines prototyping as a **development approach** promoting the implementation of a **pilot version of the intended product**. This approach is a potential solution to **the early evaluation of a system**." *IEEE*

Prototyping Dimensions

1. Representation
– Can be just textual description or can be visuals and diagrams

2. Scope
– Is it just the interface (mock-up), or does it include some computational components?

3. Executability
– Can the prototype be "run"?

4. Maturation
– What are the stages of the product as it comes along? (Throw-away vs. Evolutionary)

Types of Prototypes

Prototypes have different shapes and sizes:

- Low Fidelity vs. High Fidelity.
- Evolutionary vs. Throwaway.
- Horizontal vs. Vertical.

Low Fidelity vs. High Fidelity

Low-fidelity prototype

- Far from the final form of the system, such as paper, drawings, etc.
- Sketchy and incomplete, that has some characteristics of the target product but is otherwise simple

High-fidelity prototype

- Close to the final form of the system, much more realistic to actual application.
- A computer-based interactive representation of the product.

Throwaway vs. Evolutionary Prototyping

Throwaway** or **Rapid Prototyping:

- ✓ Creation of a model that will eventually be discarded rather than becoming part of the final delivered software.
- ✓ It can be done quickly→quick feedback
- ✓ Making changes early in the development lifecycle is **extremely cost effective.**

Evolutionary Prototyping (also known as **breadboard** prototyping):

- ✓ Build a very robust prototype in a structured manner and constantly refine it.
- ✓ Developers can focus on developing parts of the system that they understand instead of working on developing a whole system.

Horizontal vs. High Vertical

Horizontal Prototype: Provides a broad view of an entire system or subsystem, focusing on user interaction. ***Example:*** all first and second level menu commands.

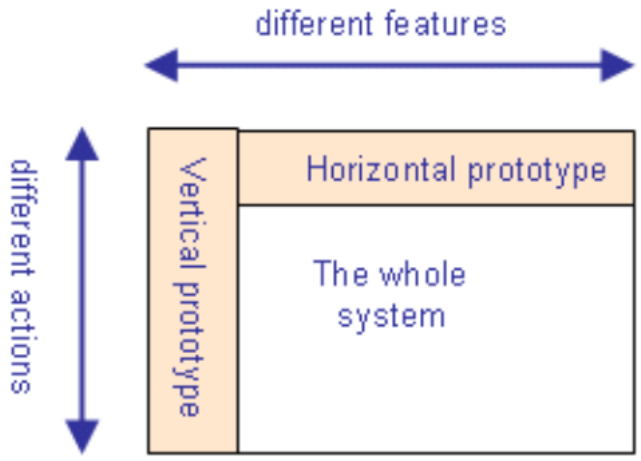
Useful for:

- Confirmation of user interface requirements and system scope,
- Develop preliminary estimates of development time, cost and effort.

Vertical Prototype: A more complete elaboration of few functions.

Useful for obtaining detailed requirements for a given function, with the following benefits:

- Refinement database design,
- Clarify complex requirements by drilling down to actual system functionality.



(14) Usability Requirements

UI vs. UX

- ***UI – User Interface***
  Look and feel.
  The thing with which the customer interacts.

- ***UX – User Experience***
  - The entire process from start to end.
  - Tries to make it pleasant for the end-user.
  - It is more analytical and includes cognitive behavior, human psychology, and UI.

What is Usability?

Usability is a *measure* of how *easy* it is to *use* something:

- How easy will the use of the software be for a *typical user* to *understand*, *learn*, and *operate*
- *e.g.*, "user-friendliness"

ISO Definition (9241-11) "...the extent to which a product can be *used* by *specified* users to achieve *specified goals* with *effectiveness*, *efficiency* and *satisfaction* in a *specified context* of use."

Why is Usability Important?

From the user's perspective: Usability can make the difference between *performing a task accurately and completely or not* and *enjoying the process or being frustrated*.

From the developer's perspective: Usability is important because it can mean the difference between the *success* or *failure* of a system.

From a management point of view: Software with poor usability *can reduce the productivity of the workforce* to a level of performance worse than without the system.

In all cases, *lack of usability can cost time and effort*.

Understanding the Context of Use

Usability is about achieving goals "with **effectiveness**, **efficiency** and **satisfaction** in a ***specified context of use***."

What is Context?: The actual **users**, **goals**, **tasks**, **equipment** (hardware, software), and **environments** (physical, social, organizational) involved.

Why Analyze It?: **Usability needs to change drastically based on context** (e.g., mobile app used outdoors vs. desktop software in a quiet office).

Key Areas:

- Who are the specific users? (Builds on user analysis)
- What are their specific goals?
- What tasks do they perform?
- Where and how will they use the software? (Environment, device)

Goals for Usability Requirements

Promote the following:

***Standardization*:** use pre-existing **industry standards** where they exist to aid learning and avoid errors e.g., Apple, and Windows interface standards

***Integration*:** theproductshouldworkwithdifferentsoftwaretoolsand packages

***Consistency*:** use common action sequences, terms, units, colors, etc. within the program compatibility across different product versions

***Portability*:** allow the user to convert data and share user interfaces across multiple software and hardware environments

Usability Factors

1. *Fit for Use*: Support tasks the user wants.

2. *Ease of Learning*: Learnability for various groups of users.

3. *Task Efficiency*: Quick tasks for frequent users.

4. *Ease of Remembering*: Remember tasks for occasional users.

5. *Subjective Satisfaction*: Overall satisfaction.

6. *Understandability*: Understand what the system or each module does.

Nielson's 10 Usability Heuristics (Design Principles)

1) ***Visibility of system status (Keep users informed)***

- A loading spinner or progress bar appears while a file is uploading.
- A confirmation message pops up after submitting a form: "Your message has been sent."

2) ***Match between system and real world (Speak the users' language)***

- A shopping cart icon is used in online stores because users associate it with buying.
- Using terms like "Inbox" and "Trash" in email apps instead of internal jargon like "msg_buffer" or "del_store."

3) ***User control and freedom (Easy 'undo' and 'exit')***

- A "Cancel" button during a multi-step process.
- Undo/redo options in Google Docs.
- The ability to exit full-screen mode with Esc.

4) ***Consistency and standards (Follow platform conventions and familiar patterns)***

- A floppy disk icon used for "Save" across many apps.
- Navigation menus on the top or left, as expected by users.
- Using blue, underlined text for hyperlinks.

5) ***Error prevention (Design carefully to prevent problems)***

- Disabling the "Submit" button until all required fields are filled.
- A confirmation dialog before deleting a file: "Are you sure you want to delete this?"
- Auto-correcting typos in a search box (e.g., Google suggesting "Did you mean...?").

6) ***Recognition rather than recall ("Minimize the user's memory load by making elements, actions, and options visible.")***

- Auto-suggestions in search bars (e.g., Google or YouTube).
- Recently used files shown in Microsoft Word.
- A dropdown menu instead of requiring the user to type a command.

7) ***Flexibility and efficiency of use (Allow shortcuts for experts)***

- Keyboard shortcuts like Ctrl + C to copy.
- Power users in Excel using formulas while beginners use the menu.

8) ***Aesthetic and minimalist design (Avoid irrelevant information)***

- Google's clean homepage with just the search bar.
- Simple mobile app designs with only essential buttons.
- A dashboard that shows only key metrics, not cluttered stats.

9) ***Help users recognize, diagnose, recover from errors (Plain language error messages)***

- "Password must contain at least 8 characters, one number, and one symbol."
- "No internet connection. Please check your network settings."
- Showing a red outline around a missing form field.

10) ***Help and documentation (Easy to search, task-focused)***

- Searchable help centers (e.g., Airbnb's or Slack's Help pages).
- Tooltips that appear when you hover over icons.

The Five *E*s of Usability

- ***Effective:*** The completeness and accuracy with which users achieve their goals.
- ***Efficient:*** The speed (with accuracy) with which users can complete their tasks.
- ***Engaging:*** The degree to which the tone and style of the interface make the product pleasant or satisfying to use.
- ***Error Tolerant:*** How well the design prevents errors or helps with recovery from those that do occur.
- ***Easy to Learn:*** How well the product supports both initial orientation and deepening understanding of its capabilities.

Making Usability Measurable

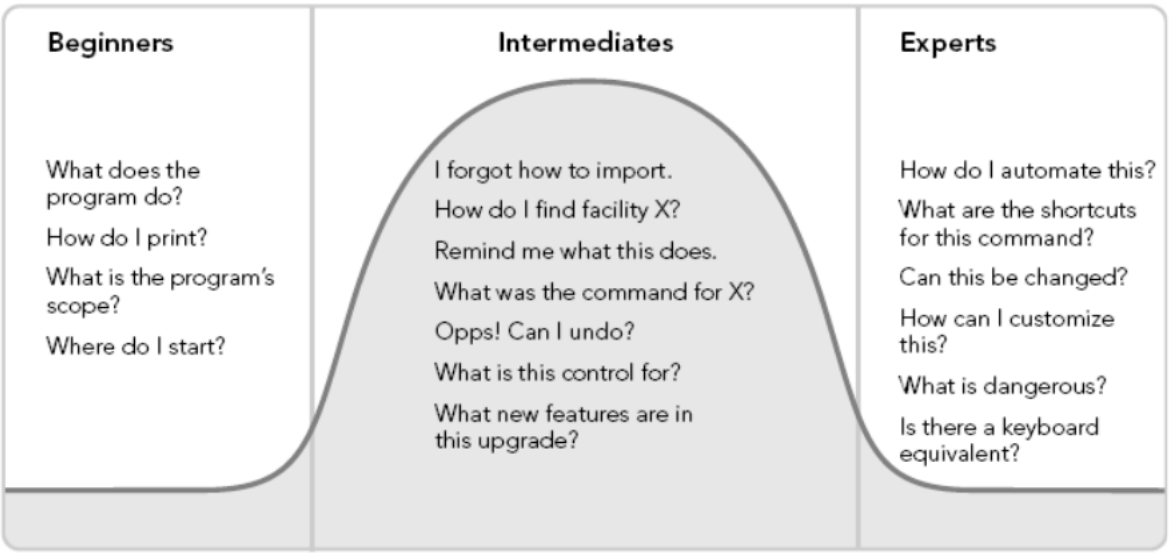
Problem: Vague requirements like "user-friendly" are hard to test.
Solution: Define specific, quantifiable criteria tied to usability goals/factors or the 5Es.

Examples:

- ***Effectiveness***:"95% of users shall successfully complete task X without error."
- ***Efficiency***:"Expert users shall take less than 45 seconds on average for task Y."
- ***Ease of Learning***:"New users shall complete the introductory tutorial Z in less than 10 minutes."
- ***Error Tolerance***:"In the event of service failure, the system shall switch to a fallback mechanism within 5 seconds in at least 98% of failure cases, without requiring user intervention."
- ***Satisfaction***:"The system shall achieve a System Usability Scale (SUS) scoreof 70+."

User analysis

- Most users are neither beginners nor experts; instead, they are intermediates.
- A well-balanced user interface does not cater to the beginner or to the expert but rather devotes the bulk of its efforts to satisfying all type of users.



UI strategies for users

Beginners	*Intermediate*	*Experts*
- Simple features. - Feedback. - Confirmation. - Tutorials.	- Help remembering - Easy to recognize, consistent - On-line help with search	- Shortcuts - Customization - Little or no feedback (distracting)

User characteristics and personas

Identify **personas** and try to classify users based on attributes/ characteristics identified:

1. Age
2. Sex
3. Culture
4. Physical abilities and disabilities
5. Education background
6. Computer/IT experience
7. Motivation
8. Attitude
9. Goals
10. Pain points

- *Identify Goals.*
- *Identify how each persona will achieve the goals.*

Accessibility: Usability for All

What: Ensuring people with diverse abilities (visual, auditory, motor, cognitive) can use the software effectively. Extends user analysis.

Importance: Ethical design, legal compliance, larger user base.

Key Considerations:

- **Visual:** Colorcontrast,fontscalability,screenreadercompatibility, alt-text for images.
- **Auditory:** Visual alternatives for audio cues, captions.
- **Motor:** Keyboard-only navigation, sufficient target sizes, tolerance for varied input strength.
- **Cognitive:** Clear layout, simple language, consistent navigation.

Standards: Reference WCAG (Web Content Accessibility Guidelines).

How Do We Evaluate Usability?

• **Purpose:** Identify usability issues, check if measurable goals are met, gather feedback.

• **Common Methods:**

- ***Inspection Methods (No End Users):***
  - **Heuristic Evaluation**: Experts check UI against heuristics.
  - **Cognitive Walkthrough**: Experts simulate user tasks step-by-step.
- ***Testing Methods (With End Users):***
  - **Think-Aloud Protocol**: Users perform tasks while verbalizing thoughts.
  - **Task Performance Analysis**: Measure time, errors, completion rates (links to measurable requirements).
  - **Surveys/Questionnaires**: Measure satisfaction (e.g., SUS).

Role of Documentation & Support

Impact on Usability: Crucial for Ease of Learning, Efficiency, and Error Tolerance.

Types:

- Tutorials(forbeginners)
- User Manuals / Guides
- Online Help / Knowledge Base (searchable)
- Context-Sensitive Help (Tooltips, pop-ups)
- Clear Error Messages (Guiding recovery)

As a Requirement: Can be specified, e.g., "Context-sensitive help must be available for all core functions."